

Data Structures — Study Notes with PYQ [Sample Version]

Course Code: B21CA04DC | **Semester:** II | **Programme:** BCA | **University:** Sreenarayanaguru Open University (SGOU)

Subject: Data Structures | **Coverage:** Blocks 1–4, 16 Units (Theory Only)

Source: Self Learning Material (SLM) — SGOU B21CA04DC

About These Notes

Prepared by: Abdul Vahab A A

Website: <https://abdulvahabaa.in>

These notes were created for my personal study and revision. I am sharing them here because someone else might find them useful too.

Wishing you all success in your semester exams. Study with focus, practise algorithms in C, and stay consistent — success will follow.

Dua mein Yaad Rakhna.

Index

Block 1: Basic Data Structures — Data & ADT, linear/nonlinear, arrays, stack & queue, Polish notations, circular queue, deque, priority queue, recursion

Block 2: Linked List — Linked allocation, operations, search & sort, circular & doubly linked lists, linked stack & queue, LL vs array

Block 3: Non-Linear Data Structures — Trees & terminologies, traversals, BST, AVL & balancing, graphs, BFS/DFS, representations & applications

Block 4: Complexity of Algorithms — Time/space complexity, asymptotics, searching & sorting, divide & conquer, backtracking, Prim's & Kruskal's MST

Quick Revision Sheets — Block 1 | Block 2 | Block 3 | Block 4 (*at the bottom*)

(Detailed subsection numbers match SGOU SLM — see each unit heading in the notes.)

Block 1: Basic Data Structures

Unit 1: Linear and Nonlinear Structures

1.1 Data and Information / Data Types / ADT

Theory

In computing, **data** is a sequence of symbols that represents an object, relationship or idea. Data represented using binary digits 0 and 1 is called **digital data**. When digital data is

processed, we obtain **digital information**. Modern computers process digital data to produce useful digital information (for example, length, breadth and height of a cube are data; volume and surface area computed from them are information).

A **data type** defines how data is represented internally in memory. Programming languages support **primitive data types** (int, char, float/real, boolean) and **derived data types** formed from primitives (pointers, structures, unions). An **Abstract Data Type (ADT)** is a mathematical model: a set of possible values together with the operations allowed on those values. For example, the set {length, breadth, height} is an ADT from which $\text{volume} = \text{length} \times \text{breadth} \times \text{height}$ can be computed.

Important Points

- Data → raw symbols/facts; Information → processed data
- Primitive types: integer, character, float/real, boolean
- Derived types: pointer, structure, union
- ADT = values + operations (implementation-independent model)
- Keywords linked to this unit: Array, Queue, Stack, Linked list, Tree, Graph

For Exam

Data is a sequence of symbols representing objects or ideas; processed data is information. A data type describes memory representation. Primitive types are language basics (int, char, float); derived types are built from them. An ADT is a mathematical model defining values and operations without specifying implementation.

Diagram (refer SLM): Fig 1.1.1 Data and Information

1.1.1 Data Structure

Theory

An **algorithm** is a finite sequence of steps to solve a problem. A **program** is an implementation of an algorithm in a programming language. During execution, programs work on data. A **data structure** is the organisation of data so that a program can use it efficiently in terms of time and space. Arrays, stacks, queues, lists, trees and graphs are common data structures used in design and analysis of algorithms.

Important Points

- Algorithm = step-by-step procedure to solve a problem
- Program = algorithm coded in a language
- Data structure = organisation of data needed to solve a problem efficiently
- Examples: Array, Stack, Queue, Linked list, Tree, Graph

For Exam

A data structure is the way we organise data so that a program can process it efficiently. It is the organisation of data needed to solve a problem. Common examples are arrays, stacks, queues, linked lists, trees and graphs.

Previously Asked Questions

- **Q10** (1 mark, Apr 2025) — Define algorithm.
 - *Answer:* An algorithm is a finite, step-by-step procedure (sequence of computational steps) used to solve a problem.

1.1.1.1 Data Structure in Everyday Life

Theory

Everyday analogies help visualise data structures. A **stack** of plates follows **LIFO** (Last-In-First-Out): only the top plate can be added or removed. A **queue** of passengers boarding a bus follows **FIFO** (First-In-First-Out): first person in line boards first; new people join at the back. A **graph** models networks: social media users are nodes and friendships are edges; Google Maps locations are nodes and roads are edges used to find shortest paths.

Important Points

Structure	Real-life analogy	Principle
Stack	Pile of plates	LIFO
Queue	Bus/ATM line	FIFO
Graph	Social network / Maps	Nodes + edges

- Graph $G = (V, E)$: V = vertices (nodes), E = edges (connections)

For Exam

Stack behaves like stacked plates (LIFO). Queue behaves like a waiting line (FIFO). Graph is a network of nodes connected by edges, used in social media and shortest-path problems like Google Maps.

Diagram (refer SLM): Fig 1.1.2 Stack as pile of plates; Fig 1.1.3 Queue; Fig 1.1.4 Graph

1.1.2 Need for Data Structures

Theory

As applications grow more complex and data volumes explode, three major problems arise. **Processor speed** alone cannot handle billions of records efficiently. **Data search** becomes slow if every item must be scanned (for example, searching one item among 10^6 inventory records). **Multiple simultaneous requests** on a web server can overload even large machines. Data structures organise data so that not every item need be searched; required data can be located quickly and space/time can be managed better.

Important Points

- Problems addressed: processor load, slow search, concurrent requests
- Goal: organise data for efficient access, insert, delete and update
- Efficiency measured mainly in **time** and **space**

For Exam

Data structures are needed because large and concurrent data loads make naive processing slow or impractical. By organising data suitably, searches and updates become faster and more space-efficient without scanning every item every time.

1.1.3 Classification of Data Structures

Theory

Data structures are broadly classified into **primitive** and **non-primitive**. Primitive structures are fundamental types supported directly by languages (integer, character, float, boolean). Non-primitive structures are user-defined / derived from primitives (lists, stacks, queues, trees, graphs) and are further divided into **linear** and **non-linear**.

Important Points

```
Data Structures
├─ Primitive (int, char, float, boolean)
└─ Non-primitive
    ├─ Linear (Array, Linked list, Stack, Queue)
    └─ Non-linear (Tree, Graph)
```

- Primitive: single type, language-supported
- Non-primitive: more complex; can store related values as one entity

For Exam

Data structures are classified as primitive (fundamental language types) and non-primitive (user-defined structures built from primitives). Non-primitive structures further split into linear and non-linear.

Diagram (refer SLM): Fig 1.1.5 Classification of data structure

1.1.4 Linear Data Structure

Theory

A data structure is **linear** if its elements are arranged in a sequential order. Elements are stored in a non-hierarchical way; each element (except the first and last) has a unique predecessor and successor. Linear structures are subdivided into **static** (fixed size) and **dynamic** (size can change at run time).

Important Points

- Sequential / non-hierarchical arrangement
- Traversal typically one-by-one from start to end
- Static linear: Array
- Dynamic linear: Linked list, Stack, Queue (size may grow/shrink)

Feature	Static (e.g. Array)	Dynamic (e.g. Linked list)
Size	Fixed in advance	Can change at run time

Feature	Static (e.g. Array)	Dynamic (e.g. Linked list)
Memory	Contiguous block	Often non-contiguous nodes
Flexibility	Low	High

For Exam

A linear data structure arranges elements sequentially so each has successors/predecessors except ends. It may be static (fixed size, e.g. array) or dynamic (resizable, e.g. linked list, stack, queue).

Previously Asked Questions

- **Q16** (2 marks, Apr 2025) — Differentiate a linear and nonlinear data structure.
 - *Answer:*

Linear	Non-linear
Elements in sequential order	Elements not in a single sequence
Each element has at most one predecessor and one successor (except ends)	An element may connect to two or more others
Non-hierarchical	Hierarchical / networked
Examples: Array, Stack, Queue, Linked list	Examples: Tree, Graph

1.1.4.1 Static Data Structure — Array

Theory

A **static data structure** has a fixed size decided in advance; memory cannot be reallocated later. The classic example is an **array**: a collection of similar-type elements stored under one name. Each element has an **index** (first index is 0 in C). Array size is the total number of elements it can hold.

Important Points

- Fixed size; known at compile/declaration time
- Array elements: same data type (char, int, float, double...)
- Index starts at 0; element at position i accessed as `arr[i]`
- Contiguous memory storage

For Exam

A static data structure has fixed size that cannot change later. An array is a static linear structure storing similar-type elements in contiguous memory, accessed by index starting from 0.

Previously Asked Questions

- **Q2** (1 mark, Apr 2025) — What is an array?
 - *Answer:* An array is a collection of similar types of data items (elements) stored in contiguous memory locations and referred to by a common name using indices.

Diagram (refer SLM): Fig 1.1.6 Basic terminology of array

1.1.4.2 Dynamic Data Structure — Linked List, Stack, Queue

Theory

In a **dynamic data structure**, size is not fixed and can change during operations. A **linked list** is a collection of nodes stored at non-contiguous locations; each node holds data and a pointer to the next node. A **stack** allows insertion and deletion only at one end called **top** (LIFO); main operations are **PUSH** and **POP**. A **queue** inserts at **rear** and deletes at **front** (FIFO); insert is **enqueue** and delete is **dequeue**.

Important Points

- Linked list: nodes + links; non-contiguous; grows/shrinks easily
- Stack: one end (top); PUSH insert, POP delete; LIFO
- Queue: two ends (front, rear); enqueue / dequeue; FIFO
- Dynamic structures suit problems where size is unknown beforehand

For Exam

Dynamic data structures can change size at run time. Linked lists use pointer-linked nodes; stacks restrict access to the top (LIFO); queues insert at rear and delete at front (FIFO).

Previously Asked Questions

- **Q3** (1 mark, Apr 2025) — What is a stack?
 - *Answer:* A stack is a linear data structure in which insertion and deletion are allowed only at one end called the top, following the LIFO (Last-In-First-Out) principle.
- **Q4** (1 mark, Apr 2025) — Define the term “queue.”
 - *Answer:* A queue is a linear data structure in which elements are inserted at one end (rear) and deleted at the other end (front), following the FIFO (First-In-First-Out) principle.
- **Q8** (1 mark, Apr 2025) — What does FIFO stands for?
 - *Answer:* First In, First Out.

Diagram (refer SLM): Fig 1.1.7 Linked list; Fig 1.1.8 Stack; Fig 1.1.9 Queue

1.1.5 Non-Linear Data Structures

Theory

In a **non-linear** data structure, elements are not arranged in a single sequence; each item may connect to two or more others. A **tree** is a multilevel hierarchical structure of **nodes**. The topmost node is the **root**; bottommost nodes are **leaves**. Trees follow a parent–child relationship: each node may have multiple children (except leaves), but at most one parent (except root). Files and folders in Windows Explorer form a tree.

A **graph** is a pictorial set of elements (**vertices**) connected by **edges**. Unlike a tree, a graph **may contain cycles**. Graphs may be **directed** (edges have direction; self-loops possible) or **undirected** (edges have no direction; traversal both ways). A vertex with no connections is an **isolated vertex**. Formally, a graph $G = (V, E)$.

Important Points

- Tree: hierarchical; no cycles; root, internal nodes, leaves
- Graph: V vertices + E edges; may have cycles
- Directed: edges leave one vertex and enter another
- Undirected: bidirectional connection
- Self-loop: edge from a vertex to itself
- Isolated vertex: degree zero / no adjacent vertices

Tree	Graph
Hierarchical parent-child	Arbitrary connections
No cycles	May have cycles
One root	No unique root required
Example: file system	Example: road network, social network

For Exam

Non-linear structures do not store elements in one sequence. Trees are hierarchical (root to leaves, no cycles). Graphs are sets of vertices linked by edges and may be directed or undirected and may contain cycles.

Diagram (refer SLM): Fig 1.1.10 Trees; Fig 1.1.11(a) Directed graph; Fig 1.1.11(b) Undirected graph

1.1.6 Contiguous and Non-Contiguous Data Structures

Theory

Regardless of complexity, data organisations fall into two storage patterns. In a **contiguous** structure, elements occupy sequential adjacent memory locations (in RAM or a file). An **array** is contiguous: each element sits next to its neighbours. In a **non-contiguous** structure, elements are scattered in memory but linked logically. A **linked list** is non-contiguous: nodes may lie anywhere, connected by pointers.

Important Points

Contiguous	Non-contiguous
Adjacent sequential storage	Scattered but linked
Example: Array	Example: Linked list
Fast index access	Flexible insert/delete
Fixed block often	Dynamic node allocation

For Exam

Contiguous structures store elements sequentially in adjacent memory (e.g. array). Non-contiguous structures scatter elements in memory and connect them by links (e.g. linked list).

Previously Asked Questions

- **Q1** (1 mark, Apr 2025) — Give an example of a contiguous data structure.
 - *Answer:* Array.

Diagram (refer SLM): Fig 1.1.12 Contiguous and Non-contiguous data structure

1.1.7 Static and Dynamic Memory Allocation

Theory

Static memory allocation is done by the compiler at **compile time**. Storage size is fixed; addresses are tracked in activation records (typically stack-managed). Variables get permanent allocation for the program's life; unused space cannot be reused and size cannot change. Execution is generally faster, but there is no memory reusability. Static allocation is typical for static structures like fixed arrays.

Dynamic memory allocation happens at **run time** from the **heap**. In C, `malloc()`, `calloc()`, `realloc()` allocate memory and `free()` releases it. Size can grow or shrink; memory can be reused when freed. It is more flexible and efficient for variable-sized structures (linked lists) but slightly slower than static allocation.

Important Points

Feature	Static	Dynamic
When allocated	Compile time	Run time
Memory area	Stack (activation)	Heap
Size change	Not possible	Possible (<code>realloc</code>)
Reusability	No	Yes (<code>free</code>)
Speed	Faster	Slower
Typical use	Fixed arrays	Linked lists, growable buffers

C functions (dynamic):

- `malloc(size)` — allocate one block of given bytes (uninitialised)
- `calloc(n, size)` — allocate n blocks; initialise to 0
- `realloc(ptr, new_size)` — resize previously allocated block (old values kept; new part garbage)
- `free(ptr)` — deallocate memory previously allocated; prevents memory wastage/leaks

For Exam

Static allocation reserves fixed memory at compile time (fast, non-reusable, used for arrays). Dynamic allocation reserves heap memory at run time using `malloc/calloc/realloc` and releases it with `free()`, allowing size changes and reuse—suited to linked lists and variable data.

Previously Asked Questions

- **Q5** (1 mark, Apr 2025) — What is the purpose of function `free()`?
 - *Answer:* The `free()` function deallocates (releases) memory that was previously allocated dynamically using `malloc()` or `calloc()`, so that the memory can be reused and wastage is reduced.
- **Q24** (2 marks, Apr 2025) — Explain static and dynamic memory allocation.
 - *Answer:* **Static memory allocation** reserves a fixed amount of memory at **compile time**. The size cannot be changed later during execution. It is fast and is used for arrays of known size. **Dynamic memory allocation** reserves memory at **run time** from the heap using functions like `malloc()`, `calloc()` and `realloc()`, and releases it with `free()`. Size can grow or shrink as needed, so it is suitable for linked lists and other variable-sized structures.

Unit 2: Array as a Data Structure

1.2.1 Array as a Data Structure

Theory

An **array** is a linear data structure that stores elements of the **same data type** under a **common name** in **contiguous** memory locations. Arrays are preferred for list and table processing. Elements are accessed by **index**. For n elements, valid indices run from **0 to $n-1$** . The lowest index is the **lower bound**; the highest is the **upper bound**. Because storage is consecutive, arrays are contiguous by nature.

Important Points

- Same type + contiguous memory + common name
- Index range: $0 \dots n-1$
- Useful for lists and tables / matrices
- Three declaration attributes: **Array Name, Array Size, Array Type**
 - Example: `int arr[10]` → name `arr`, size 10, type `int`

For Exam

An array is a linear contiguous structure storing same-type elements under one name, accessed by indices from 0 to $n-1$. Its attributes are name, size and type.

Diagram (refer SLM): Fig 1.2.1 1D eggs analogy; Fig 1.2.2 2D eggs analogy; Fig 1.2.3 Array in memory

1.2.2 Array Declaration

Theory

Array declaration is language-specific. In C the general form is:

```
data_type array_name[n1][n2]...[nn];
```

Dimensions may be 1D, 2D or multi-dimensional. Example: `int arr[5];` allocates a contiguous block for five integers. If an `int` needs 4 bytes, total memory = $5 \times 4 = 20$ bytes, reserved at compile time for a static array.

Important Points

- Syntax: `Data_type name[size];` (1D) or `Data_type name[rows][cols];` (2D)
- Size must be a positive integer (for static arrays)
- Memory = number_of_elements $\times$ sizeof(element_type)

For Exam

In C, declare arrays as `data_type name[size]` (or multiple bracket sizes for multi-D). The compiler allocates contiguous memory equal to size $\times$ element size.

1.2.3 One-Dimensional Array

Theory

A **one-dimensional (1D) array** stores identical-type elements in a single linear sequence of consecutive locations. Declaration examples: `char StudList[5];`, `int Mark[5];`.

Initialisation methods

1. **Compile-time:** values listed in braces `char StudList[5] = {'A','B','C','D','E'}; int Mark[5] = {75,80,65,85,70};`

If fewer values than size are given, remaining locations become 0.

2. **Run-time:** read with loops and `scanf()` so different runs can store different values.

Access: `array_name[index]` — e.g. `Marks[0]`, `Marks[1]`, ... up to `Marks[size-1]`.

Memory address formula (1D):

```
[
arr[i] = + i
]
```

Example: base = 100, int size = 4 $\rightarrow$ address of `Mark[2]` = $100 + 2 \times 4 = 108$.

Important Points

- Contiguous block; base address + offset
- Compile-time vs run-time initialisation
- Unspecified compile-time slots filled with 0
- Address calculation enables $O(1)$ random access

Worked example

Marks: 75, 80, 65, 85, 70 at base 100 (4-byte ints) occupy addresses 100–103, 104–107, 108–111, 112–115, 116–119 (20 bytes total).

For Exam

A 1D array stores same-type elements contiguously. Declare as `type name[n]`; initialise at compile time with braces or at run time with loops. Access by index; address of element `i` is

Base + $i \times \text{size}$.

Diagram (refer SLM): Fig 1.2.4 Schematic representation of 1D array in memory

1.2.4 Two-Dimensional Array

Theory

A **two-dimensional (2D) array** (matrix / table) stores elements in **rows and columns**. Example: `A[2][3]` has row indices 0–1 and column indices 0–2. Chessboard squares also illustrate row–column addressing.

Declaration (C): `Data_type array_name[max_rows][max_columns];`

Initialisation

- Compile-time: `int A[2][3] = {1,2,3,4,5,6};` or nested `{{1,2,3},{4,5,6}};`
- Run-time: nested `for` loops with `scanf`

Memory mapping: Computer memory is linear, so a 2D array is mapped to 1D storage using:

1. **Row-major order** — store complete row 0, then row 1, ... (C language uses row-major)
2. **Column-major order** — store complete column 0, then column 1, ...

Address formulae (0-based):

- Row-major: `Address(a[i][j]) = B + ((i × n) + j) × size` (n = number of columns)
- Column-major: `Address(a[i][j]) = B + ((j × m) + i) × size` (m = number of rows)

Access / print: use row and column indices, e.g. `A[0][1]`. Printing needs nested loops.

Important Points

- Size of 2D array = rows × columns
- Must map 2D → 1D for physical storage
- Row-major: rows contiguous; Column-major: columns contiguous
- Access: `A[row][col]`

For Exam

A 2D array stores data in rows and columns (matrix). In memory it is stored in row-major or column-major order. Address of `a[i][j]` uses base, indices, dimensions and element size. Elements are accessed and printed using nested loops.

Diagram (refer SLM): Fig 1.2.5 2D array `A[2,3]`; Fig 1.2.7–1.2.10 Row/column major ordering

1.2.5 Advantages and Disadvantages of Array

Theory

Arrays give fast indexed access and simple search/traversal; 2D arrays naturally represent matrices; many same-type values can be stored efficiently under one name. However, array size is fixed after declaration, only one data type is allowed (homogeneous), and insert/delete

in the middle is costly because elements must be shifted in contiguous memory. Allocating too much wastes space; too little causes overflow problems. Linked lists overcome several of these limits by non-contiguous dynamic nodes.

Important Points

Advantages

- Easy access by index (random access)
- Easy sequential search/traversal
- Natural for matrices (2D)
- Efficient storage of multiple similar values

Disadvantages

- Static / fixed size
- Homogeneous only
- Costly insertion/deletion (shifting)
- Possible memory wastage or shortage

Limitations leading to linked lists

- Size must be known in advance
- Cannot grow/shrink after declaration
- Shifting costly for insert/delete

For Exam

Arrays offer fast index-based access and suit lists/tables, but are fixed-size and homogeneous; insertion/deletion is difficult due to contiguous storage. Linked lists address many of these limitations.

1.2.6 Static and Dynamic Memory Allocation (Arrays)

Theory

Static array allocation fixes size at compile time: `int a[5] = {1,2,3,4,5};` — a sixth element cannot be added. **Dynamic allocation** creates/resizes storage at run time using `malloc`, `calloc`, `realloc` and releases with `free`. If an array of size 7 must grow to 10, `realloc` (or allocate-copy-free) expands capacity during execution.

Important Points

- Static: compile-time fixed size
- Dynamic: run-time flexible size via heap functions
- Same four C library functions as Unit 1: `malloc`, `calloc`, `free`, `realloc`

For Exam

Static arrays get fixed compile-time memory. Dynamic allocation uses `malloc/calloc/realloc/free` so array-like storage can change size at run time.

Previously Asked Questions

(See Unit 1 Q5 and Q24 for `free()` and static vs dynamic allocation — same concepts apply to arrays.)

Diagram (refer SLM): Fig 1.2.11 Array needing expansion

Unit 3: Stack and Queue

1.3.1 Introduction to Stack

Theory

A **stack** is a linear ADT / data structure where insertion and deletion occur only at one end called the **top**. It follows **LIFO** (Last-In-First-Out): the last item pushed is the first popped. Real-life examples: stack of coins, plates at a party. Stacks can be implemented with **arrays** (fixed capacity) or **linked lists** (growable). Compilers use stacks for recursion, expression evaluation and parenthesis checking.

Important Points

- Ordered list; access only at top
- LIFO = Last In First Out
- Implementations: 1D array or singly linked list
- Array: top pointer; empty when $TOP = -1$; full when $TOP = SIZE - 1$ (0-based)
- Linked list: $top = head$; PUSH inserts at front; POP removes from front

For Exam

A stack is a linear LIFO structure allowing insert/delete only at the top. It may be represented by an array (fixed size) or a linked list (dynamic size).

Previously Asked Questions

- **Q3** (1 mark, Apr 2025) — What is a stack? (*also covered in Unit 1*)
 - *Answer:* A stack is a linear data structure in which insertion and deletion are allowed only at one end called the top, following LIFO (Last-In-First-Out).

Diagram (refer SLM): Fig 1.3.1 Stack of coins; Fig 1.3.3 Plates; Fig 1.3.4–1.3.6 Stack structure / array / linked list

1.3.1.3 Basic Operations on Stack

Theory

Core stack operations are **PUSH** (insert at top) and **POP** (remove from top). Supporting operations: **peek/top** (read top without removal), **isFull** (overflow check), **isEmpty** (underflow check). Initialise $TOP = -1$. Before push, check full; before pop, check empty.

PUSH steps

1. Check `isFull()`; if full → overflow error, exit

2. Else $TOP = TOP + 1$
3. Store item at `stack[TOP]`
4. Return success

POP steps

1. Check `isEmpty()`; if empty → underflow error, exit
2. Else read item at `stack[TOP]`
3. $TOP = TOP - 1$
4. Return success (and the item)

PEEK: return `stack[TOP]` without changing `TOP`.

Sample in C (array stack)

```
int stack[50], TOP = -1;

void push(int item) {
    if (TOP == 49) { printf("Overflow\n"); return; }
    stack[++TOP] = item;
}

int pop() {
    if (TOP == -1) { printf("Underflow\n"); return -1; }
    return stack[TOP--];
}
```

Important Points

- Overflow: push on full stack
- Underflow: pop on empty stack
- Basic ops: PUSH, POP (+ peek, isFull, isEmpty)
- Time complexity of push/pop/peek: $O(1)$

For Exam

Stack operations: PUSH inserts at top (check overflow), POP deletes from top (check underflow), peek reads top without delete, isFull/isEmpty check status. TOP starts at -1.

Previously Asked Questions

- **Q17** (2 marks, Apr 2025) — What are the basic operations of a stack?
 - *Answer:* The basic operations are **PUSH** (insert an element at the top) and **POP** (delete/remove the top element). Additional useful operations are **peek** (view top), **isFull** and **isEmpty**.

Diagram (refer SLM): Fig 1.3.7–1.3.9 Push / pop / peek working

1.3.1.4 Applications of Stack

Theory

Stacks appear throughout system software and algorithms: undo/redo, arithmetic expression evaluation, infix $\leftrightarrow$ prefix $\leftrightarrow$ postfix conversion, syntax parsing, backtracking, string reversal and tree traversal (DFS-style). Recursion is implemented internally using a call stack.

Important Points

- Undo/Redo; Expression evaluation & conversion
- Syntax parsing; Balanced parentheses
- Backtracking; String reversal; Tree traversal
- Function call / recursion stack

For Exam

Major stack applications include expression evaluation and conversion, undo/redo, parsing, backtracking, string reversal, tree traversal and implementing recursion.

1.3.1.5 Evaluation of Arithmetic Expression / Precedence

Theory

An arithmetic expression has **operands** (variables/constants) and **operators** (+, -, *, /, ^, %, relational and Boolean operators). Evaluation order depends on **precedence** and **associativity**. Higher precedence operators evaluate first; equal precedence follows associativity (left-to-right or right-to-left). Direct infix evaluation needs repeated scanning, which is inefficient; better approach: convert to postfix/prefix, then evaluate using a stack.

Important Points — Precedence (SLM Table 1.3.1)

Precedence	Operators	Associativity
1 (highest)	() []	Left $\rightarrow$ Right
2	^ (exponent)	Right $\rightarrow$ Left
3	* / %	Left $\rightarrow$ Right
4	+ -	Left $\rightarrow$ Right
5 (lowest)	< <= > >=	Left $\rightarrow$ Right

- Example: $A + B * C - E ^ F$ evaluates by precedence: first $B * C$ and $E ^ F$, then + and - in order

For Exam

Arithmetic expressions are evaluated using operator precedence and associativity. To avoid inefficient repeated scanning of infix forms, convert to postfix/prefix and evaluate with a stack.

Diagram (refer SLM): Fig 1.3.10 Evaluation of Arithmetic Expression

1.3.1.6 Polish Notation — Infix, Prefix, Postfix

Theory

Polish notation places operators before, after, or between operands. Three equivalent forms:

1. **Infix** — operator between operands: $A+B \rightarrow \langle \text{operand} \rangle \langle \text{operator} \rangle \langle \text{operand} \rangle$
2. **Prefix (Polish)** — operator before operands: $+AB \rightarrow \langle \text{operator} \rangle \langle \text{operand} \rangle \langle \text{operand} \rangle$
3. **Postfix (Reverse Polish)** — operator after operands: $AB+ \rightarrow \langle \text{operand} \rangle \langle \text{operand} \rangle \langle \text{operator} \rangle$

Prefix/postfix remove ambiguity of precedence for machines and evaluate efficiently with stacks (no need for parentheses in fully converted forms).

Important Points

Infix	Prefix	Postfix
$A+B$	$+AB$	$AB+$
$(A-C)*B$	$*-ACB$	$AC-B*$
$A+(B*C)$	$+A*BC$	$ABC*+$

For Exam

Infix places operators between operands; prefix before; postfix after. Prefix is Polish notation; postfix is reverse Polish. Stacks convert and evaluate these forms.

1.3.1.7 Conversion of Infix to Postfix

Theory

Scan the infix expression left to right using an **operator stack** and build postfix output:

1. Operand $\rightarrow$ append to output
2. $($ $\rightarrow$ push
3. $)$ $\rightarrow$ pop/print until $($
4. Operator: while stack top has **higher** precedence (or equal with left-to-right associativity), pop to output; then push incoming operator
5. At end, pop all remaining operators

SLM example: Infix $A+B*C/(E-F)$ $\rightarrow$ Postfix $ABC*EF-/+$

Important Points

- Use stack for operators; operands go straight to output
- Parentheses force evaluation order
- Same precedence + left associativity $\rightarrow$ pop then push

More examples (SLM)

Infix	Postfix
$A+B$	$AB+$
$A+B-C$	$AB+C-$

Infix	Postfix
$(A+B)*(C-D)$	$AB+CD-*$
$A*B/C$	$AB*C/$
$2+3*4$	$234**$
$A*(B+C)/D-G$	$ABC+*D/G-$

For Exam

To convert infix to postfix, scan left to right: output operands immediately; manage operators and parentheses with a stack according to precedence and associativity; finally empty the stack onto the output.

1.3.1.8 Conversion of Infix to Prefix Expression

Theory

Standard method:

1. **Reverse** the infix expression (swap `(` and `)` while reversing)
2. Scan like postfix conversion (with precedence rules adjusted as in SLM: push if precedence $\geq$ top, else pop)
3. **Reverse** the resulting string to get prefix

SLM example: $(P+(Q*R)/(S-T))$ → Prefix $+P/*QR-ST$

Important Points

- Reverse → convert (postfix-like) → reverse again
- Parentheses swap when reversing
- Useful for stack-based evaluation of prefix (scan right-to-left)

More examples (SLM)

Infix	Prefix
$A+B$	$+AB$
$A+B-C$	$-+ABC$
$(A+B)*(C-D)$	$*+AB-CD$
$A/B*C-D+E/F/(G+H)$	$+-* / ABCD // EF + GH$
$((A+B)C-(D-E))(F+G)$	$-+ABC-DE+FG$
$A-B/(C*D^E)$	$-A/B/*CDE$

For Exam

Infix-to-prefix: reverse the expression (swap brackets), convert using a stack similar to postfix conversion, then reverse the output.

Previously Asked Questions

- **Q36** (15 marks, Apr 2025) — Describe how infix expressions are converted to prefix expressions? Convert: $(A+B)*C/(D-E)*(F+G)$

- *Answer:* Infix places the operator between operands (A+B). Prefix (Polish notation) places the operator before operands (+AB).

Conversion method:

1. Reverse the infix expression and swap opening/closing parentheses.
2. Convert this reversed expression to postfix form using an operator stack (same idea as infix-to-postfix, following precedence and associativity).
3. Reverse that result to get the prefix expression.

Given expression: $(A+B)*C/(D-E)*(F+G)$

By grouping with left-to-right associativity of $*$ and $/$:

- $(A+B) \rightarrow +AB$
- $(D-E) \rightarrow -DE$
- $(F+G) \rightarrow +FG$
- $(A+B)C \rightarrow +ABC$
- $((A+B)C)/(D-E) \rightarrow /+ABC-DE$
- full expression $\rightarrow /*+ABC-DE+FG$

So the required prefix expression is $/*+ABC-DE+FG$. (Detailed table is given below.)

Method (for answer script)

1. Reverse infix and swap parentheses.
2. Convert the reversed expression to "postfix-like" form using an operator stack (precedence/associativity).
3. Reverse that result to obtain prefix.

Worked conversion of $(A+B)*C/(D-E)*(F+G)$

Associativity of $*$ and $/$ is left-to-right, so the expression groups as:

[
 $((A+B)C)/(D-E))(F+G)$
]

Stepwise by subexpressions:

Subexpression	Prefix
$(A+B)$	$+AB$
$(D-E)$	$-DE$
$(F+G)$	$+FG$
$(A+B)*C$	$*+ABC$
$((A+B)*C)/(D-E)$	$/*+ABC-DE$
$((A+B)C)/(D-E))(F+G)$	$/+ABC-DE+FG$

Final prefix: $/*+ABC-DE+FG$

(Corresponding postfix for revision: $AB+C*DE- /FG+*$)

1.3.1.9 Evaluation of Postfix Expression

Theory

Machines evaluate postfix easily because operator order is already resolved. Algorithm:

1. Scan left to right
2. If operand → PUSH onto stack
3. If operator → POP two operands, apply operator, PUSH result
4. At end, POP final result

Worked example (SLM): Evaluate $AB+C*D/$ with $A=2, B=3, C=4, D=5$

Step	Symbol	Action	Stack
1	A	Push 2	[2]
2	B	Push 3	[2, 3]
3	+	Pop 3,2 → $2+3=5$; push 5	[5]
4	C	Push 4	[5, 4]
5	*	Pop 4,5 → $5*4=20$; push 20	[20]
6	D	Push 5	[20, 5]
7	/	Pop 5,20 → $20/5=4$; push 4	[4]

Result = 4

Important Points

- Postfix evaluates faster than infix (no precedence scanning)
- Always pop **two** operands for binary operators (first popped is right operand)
- Final stack should hold one value

For Exam

To evaluate postfix: scan left to right; push operands; on operator pop two values, compute, push result; last remaining value is the answer.

Previously Asked Questions

- **Q26** (4 marks, Apr 2025) — Explain Evaluation of Postfix Expression with an example.
 - *Answer:* To evaluate a postfix expression using a stack:
Example: Evaluate $AB+C*D/$ with $A=2, B=3, C=4, D=5$.
 1. Scan the expression from left to right.
 2. If the symbol is an operand, push it onto the stack.
 3. If the symbol is an operator, pop two operands from the stack, apply the operator (first popped is the right operand), and push the result back.
 4. After the full scan, the value remaining in the stack is the result.

- Push 2, push 3 → apply + → push 5
- Push 4 → apply * → push 20
- Push 5 → apply / → push 4
- **Result = 4** (see the step table above).

Diagram (refer SLM): Fig 1.3.11–1.3.19 Step-by-step postfix evaluation

1.3.2 Introduction to Queue

Theory

A **queue** is a linear structure with two ends: **FRONT** (deletion) and **REAR** (insertion). It follows **FIFO** (First-In-First-Out), also called LILO (Last-In-Last-Out). Real-life: ATM line, ticket window, one-way vehicle lane. OS process scheduling and time-sharing systems use queues extensively.

Important Points

- Insert at rear (enqueue); delete at front (dequeue)
- FIFO / LILO
- Open at both ends (unlike stack)
- Initialise and check empty/full before operations

For Exam

A queue is an ordered FIFO list where insertions occur at the rear and deletions at the front. The first inserted element is the first deleted.

Previously Asked Questions

- **Q4** (1 mark, Apr 2025) — Define the term “queue.” (*Unit 1/3*)
 - *Answer:* A queue is a linear data structure in which insertion is done at the rear and deletion at the front, following FIFO (First-In-First-Out).
- **Q8** (1 mark, Apr 2025) — What does FIFO stands for?
 - *Answer:* First In, First Out.

Diagram (refer SLM): Fig 1.3.2 ATM queue; Fig 1.3.20 Ticket window; Fig 1.3.21 Vehicle queue

1.3.2.1 Different Types of Queues

Theory

Queues are classified into four types:

1. **Simple Queue** — insert rear, delete front; FIFO; used in memory management, pipes, call centres, interrupts
2. **Circular Queue** — last position connects to first; better memory utilisation for fixed-size arrays

3. **Priority Queue** — each element has a priority; higher priority served first; equal priority → FIFO among them
4. **Double-Ended Queue (Deque)** — insert/delete at both ends; does **not** strictly follow FIFO

Important Points

Type	Key idea
Simple	Linear FIFO
Circular	Wrap-around; reuse freed front slots
Priority	Serve by priority
Deque	Both ends active

For Exam

Four queue types: simple (basic FIFO), circular (circular reuse of array space), priority (serve by priority), and deque (operations at both ends).

Diagram (refer SLM): Fig 1.3.22–1.3.25 Queue type representations

1.3.2.2 Basic Operations in Queue

Theory

Implemented commonly with a 1D array plus front and rear pointers. Main operations: **enqueue** and **dequeue**. Support: **peek** (see front without remove), **isFull**, **isEmpty**.

Enqueue steps

1. If full → overflow, exit
2. Else rear = rear + 1
3. Place item at queue[rear]
4. Return success

(If first insertion on empty queue, also set front appropriately.)

Dequeue steps

1. If empty → underflow, exit
2. Access item at front
3. front = front + 1
4. Return success

Sample in C (array queue)

```
int queue[50], front = -1, rear = -1;

void enqueue(int item) {
    if (rear == 49) { printf("Overflow\n"); return; }
    if (front == -1) front = 0;
```

```

    queue[++rear] = item;
}

int dequeue() {
    if (front == -1) { printf("Underflow\n"); return -1; }
    return queue[front++];
}

```

Important Points

- Enqueue @ rear; Dequeue @ front
- peek → front element
- isFull: rear reaches MAXSIZE-1 (simple array queue)
- isEmpty: no elements (often front > rear or front = -1 depending on convention)

Worked sketch

Queue holds 44, 55, 66; enqueue 77 → rear advances, Q[4]=77.

Queue holds 34, 15, 54; dequeue → remove 34, front moves to 15.

For Exam

Queue operations: enqueue inserts at rear (check overflow), dequeue removes from front (check underflow); peek, isFull and isEmpty support safe use.

Previously Asked Questions

- **Q27** (4 marks, Apr 2025) — Describe the operations performed on a queue.
 - *Answer:* A queue follows FIFO. Main operations are:
 - **Enqueue:** Insert an element at the rear. First check overflow (queue full). If not full, advance rear and store the item.
 - **Dequeue:** Delete an element from the front. First check underflow (queue empty). If not empty, take the front item and advance front.
 - **Peek / Front:** Return the front element without deleting it.
 - **isFull:** Check whether the queue has no free space.
 - **isEmpty:** Check whether the queue has no elements.

Diagram (refer SLM): Fig 1.3.26–1.3.29 Enqueue / dequeue

1.3.2.3 Applications of Queue

Theory

Queues model waiting lines in computing and daily life: railway wait-list confirmation (W/L1 served before W/L2), OS job scheduling (CPU given to jobs in arrival order), printer spooling, call-centre systems and interrupt handling.

Important Points

- Ticket / wait-list systems
- CPU / process scheduling (FCFS)
- Printer spooling; buffers; pipes
- Call centres; interrupt handling

For Exam

Queues apply wherever FIFO service is needed: OS scheduling, print spooling, wait-lists, call centres and interrupt handling.

Unit 4: Circular Queue, Double Ended Queue and Priority Queue

1.4.1–1.4.2 Introduction and Types of Queues (Revision)

Theory

Unit 4 deepens queue variants. A queue is a linear FIFO structure with front and rear; enqueue inserts, dequeue deletes. Types again: **Simple, Circular, Double-Ended (Deque), Priority**.

Important Points

- Front = deletion end; Rear = insertion end (simple/circular)
- Empty (simple array convention): FRONT = -1
- Full (simple): REAR $\geq$ N-1

Diagram (refer SLM): Fig 1.4.2 Front and Rear; Fig 1.4.3 Simple Queue

1.4.2.1 Simple Queue — Algorithms and Limitation

Theory

Simple queue: enqueue at rear, dequeue at front, FIFO. Supermarket checkout is a classic example.

QINSERT (enqueue) algorithm outline

1. If $R \geq N-1 \rightarrow$ Overflow, return
2. If $F == -1$ (empty) set $F = R = 0$; else $R = R + 1$
3. $Q[R] = \text{Item}$

QDELETE (dequeue) algorithm outline

1. If $F == -1 \rightarrow$ Underflow, return
2. $Y = Q[F]$
3. If $F == R$ (last element) set $F = R = -1$; else $F = F + 1$
4. Return Y

Limitation: After several dequeues, front slots become free, but if rear has already reached N-1 the queue reports **full** and cannot reuse those empty front cells. This wasted space

motivates the **circular queue**.

For Exam

Simple queues insert at rear and delete at front. Their array implementation wastes space after front deletions once rear hits the end—circular queues fix this.

1.4.2.2 Circular Queue

Theory

A **circular queue** connects the last position back to the first, behaving like a ring (also called **ring buffer**). It still follows FIFO, but rear/front indices advance **modulo N**, so freed slots at the beginning can be reused. Traffic lights cycling colours, print spoolers, bottle-capping lines and weekly day cycles are examples.

Enqueue (conceptual steps)

1. Check if full
2. If inserting first element, set front = 0 (or 1 per SLM indexing)
3. Increment rear circularly
4. Store item at rear

Deque (conceptual steps)

1. Check if empty
2. Take item at front
3. Increment front circularly
4. If last element removed, reset front and rear

Full condition (common): $(rear + 1) \% N == front$ or SLM style $Front == Rear + 1$ (with wrap).

Empty condition: $front == 0$ / -1 depending on convention used in the algorithm.

Important Points

- Advantage over linear queue: **better memory utilisation**
- Also called ring buffer
- Enqueue @ rear, dequeue @ front (circular index update)
- Overflow / underflow still possible when truly full/empty

For Exam

A circular queue is a FIFO queue in circular form where the last slot connects to the first, reusing empty front space. It improves memory use over a simple linear array queue.

Previously Asked Questions

- **Q9** (1 mark, Apr 2025) — What is a circular queue?
 - *Answer:* A circular queue is a linear FIFO queue implemented in circular form so that the last position connects to the first, allowing reuse of vacated front locations.

- **Q18** (2 marks, Apr 2025) — Write four examples of a circular queue in computing.
 - *Answer:* (1) Traffic light / signal control sequence, (2) Print spooler of an OS, (3) Bottle-capping systems in factories, (4) CPU round-robin / cyclic buffering (also: keyboard buffer, days-of-week style cyclic routines).

Diagram (refer SLM): Fig 1.4.1 Traffic signal; Fig 1.4.4–1.4.17 Circular queue operations walkthrough

1.4.2.3 Double-Ended Queue (Deque)

Theory

A **double-ended queue (deque / D-queue / DE-queue)** allows insertion and deletion at **both** front and rear, but not in the middle. It therefore does not strictly obey FIFO.

Variants

1. **Input-restricted deque** — insertion at **one end only** (usually rear); deletion from **both** ends
2. **Output-restricted deque** — deletion at **one end only** (usually front); insertion at **both** ends

Applications: browser history (add recent URL at front, drop oldest at back); undo lists in editors.

Important Points

Deque type	Insert	Delete
General deque	Both ends	Both ends
Input-restricted	One end	Both ends
Output-restricted	Both ends	One end

For Exam

A deque permits insert/delete at both ends. Input-restricted allows insert at one end only; output-restricted allows delete at one end only. Used in browser history and undo features.

Diagram (refer SLM): Fig 1.4.18–1.4.20 Deque variants

1.4.2.4–1.4.2.5 Priority Queue

Theory

A **priority queue** associates a **priority** with each element. Higher-priority elements are served before lower-priority ones; equal priorities are handled in arrival order. It does **not** strictly follow FIFO. Insertion may place an item in an intermediate position based on priority; deletion removes the highest-priority item (not necessarily the chronological front).

Implementations: array, multi-queue, linked list, heap tree. In **array** form:

- Method 1: insert at rear; on delete, **search** for highest priority, remove it, shift elements (slow)
- Method 2: keep queue **sorted** so highest priority stays at front; delete only from front (delete fast; insert may need ordering)

Role / examples: OS process scheduling by priority; network routers sending high-priority packets first; call-centre VIP calls.

Important Points

- Serve high priority first; same priority → FIFO among them
- Not pure FIFO
- Array / linked list / heap implementations
- OS scheduling & routers are classic uses

For Exam

A priority queue serves elements by priority rather than strict arrival order. Higher priority is processed first. Used in OS scheduling and network routers.

Previously Asked Questions

- **Q30** (4 marks, Apr 2025) — What is the role of a priority queue? Give an example.
 - *Answer:* A priority queue is a special queue in which each element has a priority. Elements are served according to priority — the highest-priority item is deleted/processed first. If two elements have the same priority, they are served in FIFO order among themselves.

Example: In an operating system, processes waiting for the CPU are kept in a priority queue. A higher-priority process is scheduled before a lower-priority one. Another example is a network router that forwards high-priority packets before normal packets.

Diagram (refer SLM): Fig 1.4.21–1.4.23 Priority queue delete methods

1.4.3 Applications of Queue (Extended)

Theory

Shared resources (printer, CPU), call-centre holding queues, interrupt handling (FCFS), ready queues in OS, semaphores, printer spooling, keyboard buffers, disk scheduling and airport runway simulation (landing/take-off sharing one runway) all use queues.

Important Points

- CPU & disk scheduling; FCFS
- Spooling; device buffers
- Interrupts; call centres
- Resource sharing among consumers

For Exam

Queues manage shared resources and waiting requests: CPU/disk scheduling, spooling, buffers, interrupts and call-centre systems.

1.4.4 Introduction to Recursion

Theory

Recursion solves a problem by solving a smaller instance of the same problem. In programming, a function that **calls itself** is recursive. Two forms: **direct recursion** (function calls itself) and **indirect / mutual recursion** (f1 calls f2 which eventually calls f1). C supports recursion; every recursive function needs an **exit/base condition** and a **progressive change** toward that base; otherwise it loops infinitely and may overflow the call stack.

Types in C

1. **Tail recursion** — recursive call is the **last** action; often convertible to iteration; compilers may optimise
2. **Non-tail recursion** — work remains after the recursive call (classic factorial often written this way)

Properties

- **Base criteria:** at least one condition stops recursion
- **Progressive approach:** each call moves closer to the base

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ with base $0! = 1$ or $1! = 1$.

Sample in C (factorial — non-tail recursion)

```
int fact(int n) {
    if (n == 0 || n == 1)    /* base case */
        return 1;
    else
        return n * fact(n - 1); /* recursive call */
}
```

Advantages: natural for recursive structures; readable algorithms; good for factorial, GCD, powers, trees.

Disadvantages: extra stack frames; slower; deep recursion may cause stack overflow; possible repeated recomputation.

Important Points

- Recursion = function calls itself (directly or indirectly)
- Must have base case + progress toward it
- Tail vs non-tail
- Implemented using system call stack (link to Unit 3 stacks)

For Exam

Recursion is the process in which a function calls itself to solve a smaller instance of the same problem until a base condition is met. It needs a base case and progressive approach; used for factorial, GCD and problems like Tower of Hanoi.

Previously Asked Questions

- **Q14** (1 mark, Apr 2025) — What is recursion in programming?

- *Answer:* Recursion in programming is the process in which a function calls itself (directly or indirectly) to solve a problem by breaking it into smaller instances of the same problem.

1.4.5 Tower of Hanoi Problem

Theory

The **Tower of Hanoi**, invented by Édouard Lucas (1883), is a classic recursive puzzle: three rods and n disks. Goal: move the entire stack from source to destination obeying:

1. Move only **one** disk at a time
2. Only move the **top** disk of a rod
3. Never place a **larger** disk on a **smaller** disk

Recursive idea for n disks (source → destination via auxiliary):

1. Recursively move $n-1$ disks source → auxiliary
2. Move largest disk source → destination
3. Recursively move $n-1$ disks auxiliary → destination

For **3 disks**, SLM shows 7 moves (minimum moves = $2^n - 1$).

Sample in C

```
void towerOfHanoi(int n, char source, char aux, char dest) {
    if (n == 1) {
        printf("Move disk 1 from%c to%c\n", source, dest);
        return;
    }
    towerOfHanoi(n - 1, source, dest, aux);
    printf("Move disk%d from%c to%c\n", n, source, dest);
    towerOfHanoi(n - 1, aux, source, dest);
}
/* Call: towerOfHanoi(3, 'A', 'B', 'C'); */
```

Important Points

- Non-numeric recursion example
- Rules: one disk; top only; no larger on smaller
- Minimum moves for n disks: $2^n - 1$
- Demonstrates divide-and-conquer via recursion

For Exam

Tower of Hanoi moves n disks between three rods under three rules, solved recursively by moving $n-1$ aside, moving the largest, then moving $n-1$ on top. Minimum moves = $2^n - 1$.

Diagram (refer SLM): Fig 1.4.24–1.4.32 Tower of Hanoi steps for three disks

Block 2: Linked List

Unit 1: Linked Allocations

2.1.1 Linked List

Theory

A data structure organises data so it can be stored and used efficiently. A list is an abstract data type; array, stack and queue are other ADTs. Arrays need contiguous memory — like theatre seats issued only by sequential seat numbers. Even if seats (memory slots) are free but not contiguous, allocation fails. A linked list solves this: elements are connected by links (pointers), not by contiguous memory locations. It is a dynamic structure that grows when items are added and shrinks when they are removed. Types include singly linked list, doubly linked list and circular linked list. In a singly linked list, navigation is forward only; each node has two parts — data and link (pointer to the next node). The first node is called the Head (or Start).

Important Points

- Array: static, contiguous memory, fixed size limit
- Linked list: dynamic, non-contiguous (scattered) memory, linked by pointers
- Types: singly, doubly, circular linked lists
- Singly linked list: forward navigation only
- Node = Data + Link; first node = Head / Start
- Link of last node = NULL (end of list)

For Exam

A linked list is a linear collection of nodes stored in non-contiguous memory, where each node holds data and a pointer to the next node. Unlike arrays, memory is allocated as needed, so free but non-contiguous slots can still be used. Singly linked lists allow only forward traversal; the first node is Head and the last node's link is NULL.

Diagram (refer SLM): Fig 2.1.1 Queue; Fig 2.1.2 Stack of books; Fig 2.1.3 Heap of stones; Fig 2.1.4 Egg rack (array); Fig 2.1.5 Node structure.

Previously Asked Questions

- **Q21** (2 marks, Apr 2025) — What are the applications of a linked list?
 - *Answer:* Linked lists are used to implement stacks, queues and trees; for dynamic memory management; in media players / games (circular lists); in navigation systems (doubly linked lists); and wherever frequent insert/delete is needed without shifting elements.

2.1.2 Representation of Linked List

Theory

To store student names (e.g. Athul, Hima, Manu, Rihan), four nodes are created. Nodes need not occupy sequential addresses. Each node's link field stores the address of the next node. The last node's link is NULL. A special pointer START holds the address of the first node so the list can be entered and traversed until NULL is found.

Important Points

- Nodes may be scattered in memory (addresses need not be sequential)
- Link of node i = address of node $i+1$
- Last node link = NULL → end of list
- START / Head stores address of the first node
- Without START, the first node cannot be located

For Exam

A linked list is represented as nodes (data + link) connected by pointers. START points to the first node; each link points to the next; NULL in the last link marks the end. Traversal starts at START and follows links until NULL.

Diagram (refer SLM): Fig 2.1.6 Four-node student names; Fig 2.1.7 Linked list with START and NULL.

2.1.3 Implementation of Linked List — Self-Referential Structure

Theory

In C, a node is implemented using a structure. A structure that contains a pointer to the same structure type is a self-referential structure. That pointer becomes the link to the next node. Data fields can be of any type and any number; the link field is always `struct node *link`.

```
struct node {  
    int data1;  
    char data2;  
    struct node *link;  
};
```

Important Points

- Self-referential structure: contains a pointer to a structure of the same type
- Used to build linked-list nodes
- Link field type: `struct node *`
- Data can be int, char, float, or multiple fields
- Memory for a node is allocated at run time (e.g. `malloc`)

For Exam

A linked-list node in C is a self-referential structure with data members and a pointer (`link`) to the same structure type. This pointer stores the address of the next node, forming the chain.

Diagram (refer SLM): Fig 2.1.8 Self-referential structures.

2.1.4–2.1.5 Algorithm and Steps to Create a Linked List

Theory

Creating a complete linked list is repeated insertion at the tail. Allocate a node, fill data, set link to NULL, attach it via START (if first) or via the previous node's link (if not), and repeat until the user stops. In C, `malloc` allocates the node; `start->data` and `start->link` initialise the first node.

Important Points

- Step 1: Create a node; collect its address
- Step 2: Store data; set link = NULL
- Step 3: If first node → store address in START
- Step 4: Else → store new node's address in previous node's link
- Step 5: Repeat until user stops
- Empty list: START = NULL
- Stacks and queues can also be built with linked lists (dynamic size)

For Exam

Creation = repeated tail insertion: allocate node → fill data → link = NULL → attach to START or previous link → repeat. START holds the first node's address; each subsequent node is linked from the previous node's link field.

Diagram (refer SLM): Fig 2.1.9 Creation of a linked list.

2.1.6–2.1.7 Traversing a Linked List

Theory

Traversal means visiting each node from the first until the end. Copy START into a temporary pointer Temp (never move START itself, or the head address is lost). Read data via Temp, then set Temp = Temp → link. Repeat until Temp becomes NULL. The same idea counts nodes or finds the last node when appending.

Important Points

- Always start from START / Head
- Use Temp so START is preserved
- Access data with arrow operator: `Temp->data`
- Advance: `Temp = Temp->link`
- Stop when Temp == NULL
- Algorithm: (1) Temp = Start (2) read data into Val (3) Temp = link of current (4) repeat until Temp is NULL

Sample in C

```

void traverse(struct node *start) {
    struct node *temp = start;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->link;
    }
}

```

For Exam

To traverse a singly linked list: copy Start into Temp; while Temp is not NULL, process Temp→data and set Temp = Temp→link. Traversal visits every node exactly once and ends when NULL is reached.

Diagram (refer SLM): Fig 2.1.10 Traversal operation in a linked list.

Previously Asked Questions

- **Q29** (4 marks, Apr 2025) — Explain the steps involved in traversing a linked list.
 - *Answer:*
 - Step 1: Copy the address of the first node from Start into a temporary pointer Temp. (Do not move Start, or the head address will be lost.)
 - Step 2: Using Temp, read/process the data of the current node (for example, print Temp→data or store it in Val).
 - Step 3: Move to the next node by setting Temp = Temp→link.
 - Step 4: Repeat Steps 2 and 3 until Temp becomes NULL; then stop.
 - In this way every node is visited exactly once from beginning to end.

Unit 2: Operations on Linked List, Search and Sort, Linked List vs Array

2.2.1 Insertion in a Linked List

Theory

Insertion places a new node at a chosen position. Three actions are always required: allocate a node, assign data, and adjust pointers. A node may be inserted at the beginning, at the end, or at a specified position between two nodes.

Important Points

- Three actions: allocate → assign data → adjust pointers
- Three cases: beginning / end / specified position
- No shifting of other elements (unlike arrays)

For Exam

Insertion in a linked list means creating a new node, storing the value, and updating links so the new node sits at the beginning, end, or a given position. Only a few pointers change; other

nodes are not shifted.

2.2.1.1 Inserting a Node at the Beginning

Theory

Create a new node with the given data. If the list is empty, set Start to the new node and its link to NULL. If not empty, set $\text{new_node} \rightarrow \text{link} = \text{Start}$, then $\text{Start} = \text{new_node}$. The old first node becomes the second.

Important Points

- Empty list: $\text{Start} = \text{new_node}$; $\text{link}[\text{new_node}] = \text{NULL}$
- Non-empty: $\text{link}[\text{new_node}] = \text{Start}$; $\text{Start} = \text{new_node}$
- Pointer updated: Start (Head)

For Exam

Insert at beginning: create node $\rightarrow$ link of new node = Start $\rightarrow$ Start = address of new node. On an empty list, Start points to the new node with link NULL.

Diagram (refer SLM): Fig 2.2.1 Inserting a node at the beginning.

2.2.1.2 Inserting at the End of the List

Theory

Create new_node with data and link = NULL. If Start is NULL, $\text{Start} = \text{new_node}$. Otherwise initialise Temp = Start and move Temp until $\text{Temp} \rightarrow \text{link}$ is NULL (last node). Then set last node's link = new_node.

Important Points

- new_node: data filled, link = NULL
- Empty $\rightarrow$ $\text{Start} = \text{new_node}$
- Non-empty $\rightarrow$ traverse to last node ($\text{link} == \text{NULL}$)
- $\text{last} \rightarrow \text{link} = \text{new_node}$

For Exam

End insertion: create node with NULL link; if list empty set Start; else traverse to the last node and set its link to the new node.

Diagram (refer SLM): Fig 2.2.2 Inserting a node at the end.

2.2.1.3 Inserting at a Specific Location (Between Two Nodes)

Theory

To insert at position POS, create the new node (Temp). Traverse to obtain prenode (node at POS-1) and postnode (node at POS). Set $\text{prenode} \rightarrow \text{link} = \text{Temp}$ and $\text{Temp} \rightarrow \text{link} = \text{postnode}$ so the new node sits between them.

Important Points

- Identify prenode (POS-1) and postnode (POS) by traversal
- prenode → link = address of new node
- new_node → link = address of postnode
- Order of pointer updates matters to avoid losing the rest of the list

For Exam

Position insertion: create node → locate previous and next nodes at POS-1 and POS → link previous to new node → link new node to the old node at POS.

Diagram (refer SLM): Fig 2.2.3 Inserting a node at a specific location.

2.2.2 Deletion from a Linked List

Theory

Deletion removes a node (by position or after searching for a value). Cases: delete first, delete last, or delete at a specified position. After unlinking, free the node's memory so it is returned to the system. Temporary pointers used during the operation should also be cleaned up when appropriate.

Important Points

- Delete beginning: Start = Start → link (or Start = NULL if only one node)
- Delete end: set second-last node's link = NULL
- Delete at POS: prenode → link = postnode (skip node at POS), then free deleted node
- Always check empty list before deleting
- Unlinked nodes must be freed (deallocated)

For Exam

Deletion unlinks a node from the chain by updating neighbouring pointers, then frees the node. First-node deletion updates Start; last-node deletion sets the previous link to NULL; middle deletion connects prenode directly to postnode.

Previously Asked Questions

- **Q5** (1 mark, Apr 2025) — What is the purpose of function free()?
 - *Answer:* `free()` deallocates dynamically allocated memory (e.g. a node created with `malloc`) and returns it to the heap so it can be reused. After deleting a linked-list node, `free()` must be called to avoid memory leaks.

2.2.2.1 Deleting from the Beginning

Theory

If Start == NULL, list is empty — cannot delete. If only one node, set Start = NULL. If more than one node, copy the link of the first node into Start (Start now points to the former second node), then free the old first node.

Important Points

- Empty → error message and stop
- Single node → Start = NULL
- Multiple → Start = Start → link; free old head

For Exam

Beginning deletion: if not empty, advance Start to the next node (or set NULL for a single-node list) and free the removed node.

Diagram (refer SLM): Fig 2.2.4 Deleting from the beginning.

2.2.2.2 Deleting from the End

Theory

If empty, stop. If one node, Start = NULL. If more nodes, traverse while keeping prenode as the previous node; when the last node is reached, set prenode → link = NULL and free the last node.

Important Points

- Need address of second-last node (prenode)
- prenode → link = NULL removes last node from the list
- Free the deleted node

For Exam

End deletion: traverse to the last node while remembering the previous node; set previous → link = NULL; free the last node.

Diagram (refer SLM): Fig 2.2.5 Deleting from the end.

2.2.2.3 Delete a Node at a Specific Location

Theory

Obtain prenode (POS-1) and postnode (POS+1) by traversal. Copy postnode's address into prenode → link, skipping the node at POS. Free the node at POS. Without freeing, the deleted node would remain in memory still pointing to the next node (memory leak / orphan node).

Important Points

- prenode → link = postnode (bypass deleted node)
- free(node at POS)
- Temporary pointers (temp, prenode, postnode) managed carefully

For Exam

Specific deletion: locate previous and next of the target → previous → link = next → free target node.

Diagram (refer SLM): Fig 2.2.6 Deleting from a specific location.

2.2.3 Searching in a Linked List

Theory

Searching traverses from Head comparing each node's data with the key. Initialise Temp = Head. If Temp → data matches, search succeeds. Else Temp = Temp → link. If Temp becomes NULL, the value is not present. Example: Head → 10 → 20 → 30 → 42 → 50 → NULL; search 42 succeeds at the fourth node.

Important Points

- Start at Head / Start
- Linear traversal with Temp
- Match → found; Temp == NULL → not found
- Binary search is generally not feasible on a singly linked list (no random access)

For Exam

Search: Temp = Head; while Temp ≠ NULL, compare Temp → data with key; on match return success; else move to next. If Temp becomes NULL, key is absent. Complexity is linear in the number of nodes.

2.2.4 Sorting in Linked List

Theory

Sorting arranges node data in ascending or descending order. Selection sort repeatedly finds the minimum (or maximum) in the unsorted part and swaps it into place. Example (descending): list 9, 11, 35, 27, 61 → after successive min-swaps with end positions → 61, 35, 27, 11, 9.

Important Points

- Any sorting algorithm can be adapted to linked lists
- Selection sort: find min in unsorted part → swap into sorted boundary → repeat
- Data values are swapped (or nodes rearranged) until the list is ordered
- No index-based random access — algorithms rely on pointer walks

For Exam

Linked-list sorting (e.g. selection sort) repeatedly finds the extreme value in the unsorted portion and places it at the correct boundary by swapping, until the entire list is ordered.

2.2.5 Linked List vs Array

Theory

Arrays store elements in contiguous compile-time (or fixed) blocks with fast index access but costly insert/delete (shifting). Linked lists allocate nodes at run time in non-contiguous memory; insert/delete mainly update pointers but access requires traversal and each node stores an extra link.

Feature	Array	Linked List
Memory layout	Contiguous locations	Non-contiguous (scattered)

Feature	Array	Linked List
Size	Fixed	Dynamic (grows/shrinks)
Allocation time	Compile time (static)	Run time (dynamic)
Memory use	Generally less (data only)	More (data + link address)
Access	Easy / random (index)	Sequential traversal only
Insertion & deletion	Slower (shifting)	Faster (pointer updates)
Search	Can use binary search if sorted	Typically linear; binary search not feasible

Important Points

- Primary advantage of linked list over array: better memory utilisation / dynamic size / no contiguous-block requirement
- Primary cost: extra pointer memory and slower random access
- Applications of linked lists: stack, queue, trees, and related dynamic structures

For Exam

Arrays are fixed-size contiguous structures with fast random access but slow insert/delete. Linked lists are dynamic, non-contiguous, pointer-linked structures with faster insert/delete but sequential access and extra memory for links. Prefer arrays for frequent random access; prefer linked lists when size varies and insert/delete are frequent.

Previously Asked Questions

- **Q37** (15 marks, Apr 2025) — Explain linked list, its operations and advantages over array.
 - *Answer:*
 - **Definition:** A linked list is a dynamic linear data structure made of nodes. Each node has a data part and a link (pointer) to the next node. Nodes need not be stored in contiguous memory. Start (or Head) points to the first node, and the last node's link is NULL.
 - **Operations:**
 - Create / insert a node (at beginning, end, or a given position)
 - Delete a node (from beginning, end, or a given position)
 - Traverse (visit each node from Start until NULL)
 - Search for a key
 - Sort the list
 - **Advantages over array:**
 - Size can grow or shrink at run time (dynamic memory)
 - No need for contiguous memory
 - Insert and delete do not require shifting other elements
 - Efficient when the number of elements is not known in advance

- **Disadvantages (brief):** No direct random access by index; extra memory is needed for link fields; traversal is sequential only.
-

Unit 3: Circular Linked List, Doubly Linked List

2.3.1 Circular Linked List

Theory

In a singly linked list the last node points to NULL. In a circular linked list the last node points back to the first node, forming a circle — there is no NULL end. From any node one can reach any other node by following links. Examples: a media player that repeats songs endlessly; multiplayer games where players sit in a circle; CPU resource allocation round-robin style.

Important Points

- Last node → first node (not NULL)
- No NULL terminator in a non-empty circular list
- From any node, every other node is reachable
- Applications: repeating media playlist, multiplayer game turns, round-robin CPU scheduling
- If the list becomes empty, the last-node pointer / Start is set to NULL

For Exam

A circular linked list is like a singly linked list except the last node's link points to the first node, forming a loop. Traversal can start anywhere and continues until it returns to the starting node. It has no NULL end marker while non-empty.

Diagram (refer SLM): Fig 2.3.1 Ordinary linked list; Fig 2.3.2 Circular linked list.

Previously Asked Questions

- **Q28** (4 marks, Apr 2025) — Write short notes on circular and doubly linked lists.
 - *Answer:*
 - **Circular linked list:** It is like a singly linked list, but the last node's link points back to the first node instead of NULL, forming a circle. There is no NULL end while the list is non-empty. Traversal can start from any node and continues until it returns to the starting node. It is useful for round-robin CPU scheduling, repeating media playlists, and multiplayer turn-based games.
 - **Doubly linked list:** Each node has three parts — data, left (previous) link, and right (next) link. So we can move both forward and backward. The left link of the first node and the right link of the last node are NULL. It needs more memory than a singly linked list because of two pointers, but insert/delete and navigation are easier in both directions. It is used in navigation systems and applications needing bidirectional traversal.

2.3.1.1 Creation of Circular Linked List

Theory

Create the first node in Start; set End = Start. For each new node Temp: read data, End → link = Temp, End = Temp. After all nodes are added, set Temp → link (last) = Start so the circle closes.

Important Points

- Start and End initially point to the first node
- Append via End → link = Temp; End = Temp
- Final step: last → link = Start
- Stop when construction finishes

For Exam

Circular list creation: build nodes as in a linear list using Start and End, then make the last node's link point to Start instead of NULL.

2.3.1.2 Traversing a Circular Linked List

Theory

Set Temp = Start. If Temp is NULL, stop (empty). Otherwise process data and advance Temp = Temp → link. Continue until the condition for completing one full circle is met (typically until returning to Start / until Temp → link == Start as per SLM algorithm), then stop. Unlike singly lists, do not wait for NULL.

Important Points

- Temp = Start initially
- Empty check: Temp == NULL
- Advance with Temp = Temp → link
- Stop when circle completes (back at Start), not on NULL
- Advantage: start from any node and still cover all nodes

For Exam

Circular traversal: begin at Start (or any node), visit each node by following links, and stop when you return to the starting node. There is no NULL end check as in a singly linked list.

2.3.2 Structure of Doubly Linked List

Theory

A doubly linked list allows navigation forward and backward. Each node has three fields: Data, Left Link (previous / predecessor), and Right Link (next / successor). The left link of the first node is NULL; the right link of the last node is NULL. From any node, both successor and predecessor are directly accessible.

Important Points

- Three fields: Left link | Data | Right link
- Bidirectional navigation

- First → leftLink = NULL; Last → rightLink = NULL
- More space than singly linked list (two pointers per node)
- Insert/delete need more pointer updates but are flexible in both directions
- Application: navigation systems (move next/previous)

For Exam

A doubly linked list is a two-way list where each node stores data plus pointers to the previous and next nodes. This enables forward and backward traversal. Ends are marked with NULL on the outer links.

Diagram (refer SLM): Fig 2.3.3 Node of a doubly linked list; Fig 2.3.4 A doubly linked list.

Previously Asked Questions

- **Q11** (1 mark, Apr 2025) — What is the purpose of a doubly linked list?
 - *Answer:* To allow traversal and navigation in both forward and backward directions by storing previous and next pointers in each node.

2.3.3 Algorithm for Creation of a Doubly Linked List

Theory

Create First; set First → leftLink = NULL; read data. Keep back = First. For each new Far node: read data; back → rightLink = Far; Far → leftLink = back; back = Far. Finally Far → rightLink = NULL.

Important Points

- First node: leftLink = NULL
- Cross-link every new node both ways (right of previous, left of new)
- Last node: rightLink = NULL
- back always tracks the current last node during construction

For Exam

DLL creation links each new node with both rightLink of the previous and leftLink of the new node, leaving NULL on the outer ends of the first and last nodes.

2.3.3.1 Insertion of a Node Between Two Nodes (DLL)

Theory

Given prev_node, insert new_node after it: set new_node → data; new_node → next = prev_node → next; prev_node → next = new_node; new_node → prev = prev_node; if new_node → next ≠ NULL then new_node → next → prev = new_node.

Important Points

- Update four relationships: new ↔ prev and new ↔ old next
- Order: attach new's next first, then prev's next, then new's prev, then next's prev

- Number of pointers affected is greater than in a singly linked list

For Exam

DLL middle insert: create node $\rightarrow$ point new $\rightarrow$ next to prev $\rightarrow$ next $\rightarrow$ prev $\rightarrow$ next = new $\rightarrow$ new $\rightarrow$ prev = prev $\rightarrow$ if next exists, next $\rightarrow$ prev = new.

Diagram (refer SLM): Fig 2.3.5 Insertion of new node between two nodes.

2.3.4 Deletion of a Node in Doubly Linked List

Theory

Let del be the node to delete. If del is head, move head to del $\rightarrow$ next. If del $\rightarrow$ next exists, set del $\rightarrow$ next $\rightarrow$ prev = del $\rightarrow$ prev. If del $\rightarrow$ prev exists, set del $\rightarrow$ prev $\rightarrow$ next = del $\rightarrow$ next. Then free(del).

Important Points

- Bypass del from both sides using prev and next
- Special case: deleting head updates head pointer
- Always free(del) after unlinking
- More pointer fixes than singly linked deletion, but no need to search for previous from Start (prev is stored)

For Exam

DLL deletion: reconnect neighbours (next $\rightarrow$ prev and prev $\rightarrow$ next), update head if needed, then free the deleted node.

Diagram (refer SLM): Fig 2.3.6 Deletion; Fig 2.3.7 After deletion of middle node.

Unit 4: Linked List Representation of Stack and Queue

2.4.1 Linked List Representation of Stack

Theory

Array stacks need a fixed maximum size decided in advance. A linked-list stack grows as nodes are allocated, so size need not be fixed. The top of the stack is the first node of the list; top always points to the most recently pushed item. Push inserts at the front; pop removes from the front. No large data movement is required — only pointer updates and allocation/deallocation.

Important Points

- Top = first node (most recently inserted)
- Dynamic size — allocate on push, free on pop
- LIFO preserved: push/pop at the same end (front)
- Advantage over array stack: no overflow from fixed capacity (until memory exhausts)
- Empty stack: top / Start = NULL

For Exam

In a linked stack, Top points to the first node. Push creates a node and inserts it at the beginning; pop removes the beginning node and advances Top. Size is dynamic and follows LIFO.

Diagram (refer SLM): Fig 2.4.1 Linked list representation of stack.

2.4.1.1 Push Operation

Theory

Create a new node. If the stack is empty, store data, set link = NULL, and make it the start/top. If not empty, set new → link = current top (start), then make the new node the start/top. Example: stack top = 3 then 5; push 11 → new top is 11 linking to 3.

Important Points

1. Create node
2. Empty → node becomes start; link = NULL
3. Non-empty → new → link = start; start = new (insert at beginning)
4. Top always becomes the new node after push

For Exam

Push (linked stack): allocate node, set data, link it to the current top, then update top to the new node — equivalent to insertion at the beginning of a singly linked list.

Diagram (refer SLM): Fig 2.4.2 PUSH operation using linked list.

2.4.1.2 Pop Operation

Theory

If top is NULL, stack is empty — cannot pop (underflow). Otherwise store the top node temporarily, set top = top → link, return/process the popped value, and free the old top node.

Important Points

1. If top == NULL → empty / error
2. Save top in temporary pointer
3. top = top → next
4. Return popped value; free temporary node

For Exam

Pop: if not empty, advance top to the next node and free the old top node. Removes the most recently pushed element (LIFO).

Diagram (refer SLM): Fig 2.4.3 POP operation using linked list.

2.4.2 Linked List Representation of Queue

Theory

A linked queue uses two pointers: front (first node) and rear (last node). Insertion (enqueue) happens at rear; deletion (dequeue) happens at front — FIFO. In an empty queue both front and rear are NULL. Linked representation avoids the fixed-size limitation of array queues.

Important Points

- front → first element; rear → last element
- Empty: front = rear = NULL
- Enqueue at rear; dequeue at front
- FIFO order preserved
- Dynamic allocation per enqueue

For Exam

A linked queue is a list with front and rear pointers. Enqueue adds after rear and updates rear; dequeue removes the front node and updates front. Empty when both pointers are NULL.

2.4.2.1 Insertion or Enqueue

Theory

Allocate node p, set info(p) = x, next(p) = NULL. If rear == NULL (empty), set front = p (and rear = p). Else set next(rear) = p and rear = p. The new element becomes the last node.

Important Points

- Always insert at rear
- Empty queue: front = rear = new node
- Non-empty: rear → link = new; rear = new
- new → link = NULL

For Exam

Enqueue: create node with NULL link; if queue empty set front and rear to it; else link it after rear and move rear forward.

Diagram (refer SLM): Fig 2.4.4 Insertion (enqueue) using linked list.

2.4.2.2 Deletion or Dequeue

Theory

If FRONT == NULL, underflow — cannot delete. Else PTR = FRONT; FRONT = FRONT → NEXT; FREE PTR. The former second node becomes the new front. Example: queue 8 → 5 → ...; after dequeue of 8, front points to 5.

Important Points

1. IF FRONT = NULL → Underflow; stop
2. PTR = FRONT

3. FRONT = FRONT → NEXT

4. FREE PTR

5. (If queue becomes empty, rear should also be set NULL in a complete implementation)

For Exam

Dequeue: if front is not NULL, save front, move front to the next node, and free the old front node — removes the earliest inserted element (FIFO).

Diagram (refer SLM): Fig 2.4.5 (a) Linked queue with three nodes; (b) after deletion of front node.

Block 3: Non-Linear Data Structures

Unit 1: Trees

3.1.1 Tree — Concepts and Terminologies

Theory

Trees are hierarchical **non-linear** data structures made of **nodes** connected by **edges**. Unlike arrays, stacks, queues and linked lists (linear structures), elements in a tree do **not** form a single sequence — there is no unique predecessor and unique successor for every element.

Each tree has a **root** (starting point). Every other node is linked by edges forming a **parent-child** relationship. Real-life examples from the SLM: travel options from Cochin to Mumbai (By Sea / Road / Air / Rail), and a computer file-folder hierarchy (**course** → Mechanical, Computer, Electrical, Electronics → B.Tech, BCA).

Important Points

- Linear DS: sequential storage; unique predecessor & successor (arrays, linked lists, stacks, queues)
- Non-linear DS: non-sequential; no unique pred/succ (trees, graphs)
- Tree = nodes + edges; edges show parent-child relation
- Every tree has exactly one root (no parent)
- Examples: organisation charts, file systems, expression trees, decision trees

For Exam

A tree is a hierarchical non-linear data structure consisting of nodes connected by edges. It has one root node and every other node is reached from the root through parent-child links. Data is stored in a distributed (non-sequential) manner, so there is no unique predecessor or successor for each element.

Diagram (refer SLM): Fig 3.1.1(a)–(b) Classification / types of DS; Fig 3.1.2 Structure of tree; Fig 3.1.3 Travel options Cochin → Mumbai; Fig 3.1.4 File-folder hierarchy.

3.1.1.1 Tree — Key Terminologies

Theory

From a typical tree (SLM Fig 3.1.5 with root A and nodes B...L), the following terms are defined.

Term	Definition (as in SLM)	Example (Fig 3.1.5)
Node	Entity holding data; connected by edges	A, B, C, ... L
Root	Node with no parent	A
Leaf / Terminal	Node with no children	E, G, H, I, J, K, L
Child	Node linked downward from a parent	Children of A: B, C, D
Parent	Immediate ancestor of a node	Parent of F is B
Siblings	Nodes sharing the same parent	B, C, D are siblings
Height	Number of nodes on the longest path from root to a leaf	Path A-B-F-K → height = 4
Depth	Length of path from root to that node; depth of root = 0	Depth of G = 2; of L = 3
Degree of a node	Number of children of that node	Deg(C)=1, Deg(D)=2
Degree of a tree	Degree of the node with maximum children	Deg(tree)=3 (nodes A and B)
Subtree	Tree formed by a node and all its descendants	Left/right subtrees of root

Important Points

- Root has no parent; leaf has no children
- Height counts nodes on longest root-to-leaf path (SLM convention)
- Depth(root) = 0
- Degree(node) = count of children; Degree(tree) = max degree among nodes
- Siblings = same parent
- **PYQ Q6 For Exam:** Degree of a node = number of children of that node. Degree of a tree = maximum degree among all nodes.

For Exam

Key tree terms: **Root** — no parent. **Leaf** — no children. **Height** — nodes on longest root-to-leaf path. **Depth** — path length from root (root depth = 0). **Degree of a node** — number of children. **Degree of a tree** — max children any node has. **Siblings** — nodes with the same parent.

Previously Asked Questions

- **Q6** (1 mark, Apr 2025) — Define degree of a node in tree?
 - *Answer:* The degree of a node in a tree is the number of children that node has.

Diagram (refer SLM): Fig 3.1.5 The Tree and its related terms.

3.1.2 Binary Trees

Theory

In a general tree a node may have any number of children. In a **binary tree**, each node has **at most two** children (left and right), or the tree may be empty.

A non-empty binary tree consists of:

1. A **root** node
2. A **left subtree** (itself a binary tree)
3. A **right subtree** (itself a binary tree)

Five types (SLM): Complete, Full, Perfect, Balanced, 2-tree (Extended).

Important Points

- Binary tree: 0, 1 or 2 children per node (max 2)
- Empty tree is also a binary tree
- Left and right positions matter (ordered)

For Exam

A binary tree is a finite set of nodes that is either empty or consists of a root with two disjoint binary trees — left and right subtrees. Each node has at most two children.

Diagram (refer SLM): Fig 3.1.6 Generic Binary Tree.

3.1.2.1 Complete Binary Trees

Theory

In a **complete binary tree**, all levels are completely filled except possibly the last; nodes on the last level are filled **from left to right**.

Important Points

- All levels full except last
- Last level: nodes as far **left** as possible
- Used in heap implementations

For Exam

A complete binary tree has every level fully filled except possibly the last, and all nodes in the last level appear as far left as possible.

Diagram (refer SLM): Fig 3.1.7 Complete Binary Tree.

3.1.2.2 Full Binary Trees

Theory

In a **full binary tree**, every node other than leaves has **exactly two** children. Equivalently: every node has **0 or 2** children (never exactly one).

Important Points

- No node has only one child

- Complete $\neq$ Full (complete allows last-level partial fill; full requires 0 or 2 children everywhere)
- A complete tree is “full except possibly last level, left-aligned”

For Exam

A full binary tree is one in which every internal node has exactly two children and every leaf has none — each node has either 0 or 2 children.

Diagram (refer SLM): Fig 3.1.8 Full Binary Trees.

3.1.2.3 Perfect Binary Tree

Theory

In a **perfect binary tree**, every internal node has two children and **all leaves are at the same level**.

Important Points

- All levels completely filled
- Number of nodes = $2^h - 1$ if height is counted in levels carefully; structure is fully symmetric

For Exam

A perfect binary tree has all internal nodes with two children and all leaves at the same level — every level is completely filled.

Diagram (refer SLM): Fig 3.1.9 Perfect binary tree.

3.1.2.4 Balanced Binary Tree

Theory

A binary tree is **balanced** if for every node, the heights of left and right subtrees differ by **at most 1**.

- Fig 3.1.10(a): heights differ by 1 → **balanced**
- Fig 3.1.10(b): heights differ by 2 → **unbalanced**

Important Points

- Balance condition applies at **every** node
- Difference of heights ≤ 1
- Improves search efficiency (links to Unit 3 AVL)

For Exam

A balanced binary tree is one in which the height of the left and right subtrees of every node differs by at most one.

Diagram (refer SLM): Fig 3.1.10(a) Balanced; Fig 3.1.10(b) Unbalanced.

3.1.2.5 2-Tree or Extended Binary Trees

Theory

A **2-tree (extended binary tree)** has the property that each node has either **0 children or exactly 2 children**.

Important Points

- Same 0-or-2 rule as full binary tree in SLM sense
- Example: all nodes except a leaf G have two children

For Exam

A 2-tree or extended binary tree is a binary tree in which every node has either no children or exactly two children.

Diagram (refer SLM): Fig 3.1.11 Extended Binary Tree.

3.1.3 Structure of a Node

Theory

Each binary-tree node stores:

1. **Data** (information)
2. Pointer / edge to **left child**
3. Pointer / edge to **right child**

Typically implemented as a linked structure:

```
struct node {  
    int data;  
    struct node *leftChild;  
    struct node *rightChild;  
};
```

Important Points

- Three parts: data + left address + right address
- NULL pointers mean missing child / empty subtree
- All nodes share the same structure

For Exam

A binary tree node contains a data field and two pointers — one to the left child and one to the right child. NULL indicates absence of a child.

Diagram (refer SLM): Fig 3.1.12 Node of a binary tree.

3.1.4 Tree Traversal

Theory

Traversal means visiting every node of a tree (often printing values). Uses: search a node, process some/all nodes.

For each node three actions are possible: **V** = process node, **L** = visit left, **R** = visit right.

Naming depends on when **V** occurs:

Traversal	Order	Mnemonic
Inorder	$L \rightarrow V \rightarrow R$	Left, Root, Right
Preorder	$V \rightarrow L \rightarrow R$	Root, Left, Right
Postorder	$L \rightarrow R \rightarrow V$	Left, Right, Root

SLM travel analogy (Thiruvananthapuram = root, Cochin = left, Kozhikode = right):

- Inorder $\approx$ Cochin, Thiruvananthapuram, Kozhikode
- Preorder $\approx$ Thiruvananthapuram, Cochin, Kozhikode
- Postorder $\approx$ Cochin, Kozhikode, Thiruvananthapuram

Important Points

- Visit all nodes exactly once in a defined order
- Binary tree: max 2 children $\rightarrow$ three classical orders
- Recursive algorithms naturally match L/V/R

For Exam

Tree traversal visits all nodes systematically. Three standard binary-tree traversals are inorder (LVR), preorder (VLR) and postorder (LRV), named according to when the root is processed relative to left and right subtrees.

Diagram (refer SLM): Fig 3.1.13 Kerala Map analogy.

3.1.4.1 Inorder Traversal

Theory

Order: Left subtree $\rightarrow$ Root $\rightarrow$ Right subtree (**L-V-R**).

Algorithm (recursive):

```
void inorder(struct node *tree) {
    if (tree == NULL)
        return;
    inorder(tree->left);
    printf("%d ", tree->data);    /* process root */
    inorder(tree->right);
}
```

Worked example (SLM Fig 3.1.14 / 3.1.15):

Inorder result:

5, 3, 6, 2, 4, 1, 8, 7, 10, 9, 11

Explanation sketch: leftmost leaf 5, then 3, then 6; then 2 and 4; then root 1; then right subtree 8, 7, 10, 9, 11.

Important Points

- L then V then R
- For a **BST**, inorder yields **sorted ascending** keys (Unit 2)

For Exam

Inorder traversal processes the left subtree, then the root, then the right subtree (L-V-R).

Example sequence from SLM: 5 3 6 2 4 1 8 7 10 9 11.

Diagram (refer SLM): Fig 3.1.14 Binary Tree; Fig 3.1.15 Inorder traversal.

3.1.4.2 Preorder Traversal

Theory

Order: Root → Left subtree → Right subtree (**V-L-R**).

Algorithm:

```
void preorder(struct node *tree) {
    if (tree == NULL)
        return;
    printf("%d ", tree->data); /* process root first */
    preorder(tree->left);
    preorder(tree->right);
}
```

Worked example (SLM Fig 3.1.16):

Preorder result:

1, 2, 3, 5, 6, 4, 7, 8, 9, 10, 11

Root 1 first, then entire left subtree (2, 3, 5, 6, 4), then right (7, 8, 9, 10, 11).

Important Points

- Root visited first — useful for copying/prefix expression trees
- V then L then R

For Exam

Preorder traversal processes the root first, then the left subtree, then the right subtree (V-L-R).

Example: 1 2 3 5 6 4 7 8 9 10 11.

Diagram (refer SLM): Fig 3.1.16 Preorder traversal.

3.1.4.3 Postorder Traversal

Theory

Order: Left subtree → Right subtree → Root (**L-R-V**).

Algorithm:

```
void postorder(struct node *tree) {  
    if (tree == NULL)  
        return;  
    postorder(tree->left);  
    postorder(tree->right);  
    printf("%d ", tree->data);    /* process root last */  
}
```

Worked example (SLM Fig 3.1.17 / 3.1.18):

Postorder result:

5, 6, 3, 4, 2, 8, 10, 11, 9, 7, 1

Children before parents; root 1 is last.

Important Points

- Root last — useful for deleting trees / postfix expressions
- L then R then V
- **PYQ Q25 For Exam:** Explain inorder (LVR), preorder (VLR), postorder (LRV) with a common binary-tree example and list the three sequences. Mention use of inorder on BST for sorted order.

For Exam

Postorder traversal processes left subtree, then right subtree, then the root (L-R-V). Example: 5 6 3 4 2 8 10 11 9 7 1. Together with inorder and preorder, these are the three classical binary-tree traversals asked in exams.

Previously Asked Questions

- **Q25** (2 marks, Apr 2025) — Explain different tree traversals with examples.
 - *Answer:* Tree traversal means visiting every node of a binary tree systematically. There are three standard methods:
 - **Inorder (LVR):** Left subtree → Root → Right subtree. Example: 5, 3, 6, 2, 4, 1, 8, 7, 10, 9, 11
 - **Preorder (VLR):** Root → Left subtree → Right subtree. Example: 1, 2, 3, 5, 6, 4, 7, 8, 9, 10, 11
 - **Postorder (LRV):** Left subtree → Right subtree → Root. Example: 5, 6, 3, 4, 2, 8, 10, 11, 9, 7, 1

Diagram (refer SLM): Fig 3.1.17 / Fig 3.1.18 Postorder traversal.

Unit Recap (Unit 1 — from SLM)

- Trees = hierarchical non-linear DS (nodes + edges)

- Root, leaf, height, depth, degree, siblings
 - Binary tree types: complete, full, perfect, balanced, 2-tree
 - Node = data + left + right pointers
 - Traversals: Inorder LVR, Preorder VLR, Postorder LRV
-

Unit 2: Binary Search Tree

3.2.1 Introduction to Binary Search Tree

Theory

A **Binary Search Tree (BST)** is a special binary tree used for **efficient searching**. Keys are arranged so that:

- Every key in the **left subtree** of a node is **less than** the node's key
- Every key in the **right subtree** is **greater than** the node's key

SLM example: district IDs as keys — Cochin=7, Thiruvananthapuram=1, Kozhikode=11 form a small BST; inorder gives **1, 7, 11** (ascending). Larger Kerala district-ID tree (root 8) inorders to **1...14** sorted.

Important Points

- $BST \subset$ binary trees (extra ordering property)
- Keys must be **unique** (SLM)
- Property must hold at **every** node
- Inorder traversal of BST → **sorted ascending** list
- Searching is faster than in an ordinary binary tree because comparisons discard half the tree each step (when balanced)

For Exam

A binary search tree is a binary tree in which for every node, all keys in the left subtree are smaller than the node's key and all keys in the right subtree are larger. Inorder traversal produces keys in ascending order, enabling binary-search-like lookup.

Diagram (refer SLM): Fig 3.2.1 Small BST; Fig 3.2.2 District-ID BST; Fig 3.2.3 BST example.

3.2.1.1 Properties of Binary Search Tree

Theory

Formal properties:

1. Keys are **unique**
2. Left subtree keys $<$ root key
3. Right subtree keys $>$ root key
4. Left and right subtrees are themselves BSTs

Example inorder (Fig 3.2.3): **83 85 90 104 108 109 110 117 125 128** — sorted.

- **PYQ Q20 For Exam:** A binary tree has no ordering restriction (any arrangement of ≤ 2 children). A BST additionally enforces $\text{left} < \text{node} < \text{right}$ at every node, unique keys, and inorder sorted order — enabling efficient search/insert/delete.

Important Points

Feature	Binary Tree	Binary Search Tree
Children	At most 2	At most 2
Key order	No rule	$\text{Left} < \text{node} < \text{Right}$
Inorder result	Any order	Ascending sorted
Search	May scan all nodes	Compare & go left/right
Keys	May repeat (general)	Unique (SLM)

For Exam

BST differs from a general binary tree by the search-tree property: left keys smaller, right keys larger, unique keys, and recursive BST subtrees. Hence inorder lists keys in sorted order and search follows a single path.

Previously Asked Questions

- **Q20** (2 marks, Apr 2025) — How does a binary search tree differ from a binary tree?
 - *Answer:* A binary tree has no ordering rule on keys (any arrangement of at most two children). A BST additionally requires that for every node, left subtree keys are smaller and right subtree keys are larger, keys are unique, and inorder traversal gives sorted ascending order — enabling efficient search/insert/delete.

3.2.2 Searching in a Binary Search Tree

Theory

Compare target `val` with root:

- Equal → **success**
- `val < root` → search **left**
- `val > root` → search **right**
- Reach NULL → **failure**

Algorithm (recursive):

```
struct node *searchBST(struct node *tree, int val) {
    if (tree == NULL)
        return NULL;           /* failure */
    if (tree->data == val)
        return tree;           /* success */
    else if (val < tree->data)
        return searchBST(tree->left, val);
    else
```

```

        return searchBST(tree->right, val);
    }

```

Worked example (Fig 3.2.4): Search **76**. Root 57 → 76 > 57 → go right; continue until 76 found.

Important Points

- Search = walk one path root → leaf (or hit)
- Best when tree is balanced; worst (skewed) ≈ linear list
- Main application of BST is searching

For Exam

To search in a BST, compare the key with the root. If equal, stop; if smaller, recurse left; if larger, recurse right. Failure occurs if a NULL pointer is reached. Example: searching 76 starts at 57 and moves right until found.

Diagram (refer SLM): Fig 3.2.4 Searching a BST.

3.2.3 Insertion Operation in BST / Construction

Theory

Insertion is essentially a **failed search**: follow left/right until NULL, then attach the new node there. If key already exists → insertion **fails** (unique keys).

Algorithm idea:

1. If tree empty → create root with item
2. If item equals current → failure
3. If item < current → insert in left subtree
4. Else → insert in right subtree

Sample in C

```

struct node *insertBST(struct node *tree, int item) {
    if (tree == NULL) {
        tree = (struct node *)malloc(sizeof(struct node));
        tree->data = item;
        tree->left = tree->right = NULL;
    }
    else if (item < tree->data)
        tree->left = insertBST(tree->left, item);
    else if (item > tree->data)
        tree->right = insertBST(tree->right, item);
    return tree;
}

```

Worked construction example (exam-style):

Insert keys in order: **50, 30, 70, 20, 40, 60, 80**

Step 1: 50 → root
50

Step 2: 30 < 50 → left
50
/
30

Step 3: 70 > 50 → right
50
/ \
30 70

Step 4: 20 < 50, < 30 → left of 30
50
/ \
30 70
/
20

Step 5: 40 < 50, > 30 → right of 30
50
/ \
30 70
/ \
20 40

Step 6: 60 > 50, < 70 → left of 70
50
/ \
30 70
/ \
20 40 60

Step 7: 80 > 50, > 70 → right of 70
50
/ \
30 70
/ \
20 40 60 80

Inorder of final tree: **20 30 40 50 60 70 80** (sorted — confirms BST).

SLM illustration: Insert **51** — search ends at dead-end near 49/50; attach as right child of 50 when that child is NULL (Fig 3.2.5–3.2.6).

- **PYQ Q31 For Exam:** Explain construction by successive insertion: empty tree → root; then for each key, search to NULL leaf position and link left if smaller / right if larger; reject duplicates. Give a step-by-step numeric example and note that inorder is sorted.

Important Points

- Insert = search until NULL, then attach
- Duplicate → fail
- New node always becomes a **leaf** initially
- Order of insertion changes shape (same set can give different trees)

For Exam

A BST is constructed by inserting keys one by one. The first key becomes the root. Each later key is compared with nodes from the root downward until a NULL child link is found; it is inserted as a left child if smaller than the parent, otherwise as a right child. Existing keys are not re-inserted. Example construction of 50,30,70,20,40,60,80 yields a tree whose inorder is 20...80.

Previously Asked Questions

- **Q31** (4 marks, Apr 2025) — Explain how a binary search tree is constructed.
 - *Answer:* A BST is built by inserting keys one by one using the BST property (left child < parent < right child).
- Example:** Insert 50, 30, 70, 20, 40, 60, 80.
1. The first key becomes the root.
 2. For each new key, start from the root and compare:
 - if key < current node → go to the left subtree
 - if key > current node → go to the right subtree
 3. Continue until a NULL link is found, then insert the new node there as a leaf.
 4. If the key already exists, insertion fails (keys are unique).
 - 50 is root; 30 goes left; 70 goes right; 20 left of 30; 40 right of 30; 60 left of 70; 80 right of 70.
 - Inorder traversal of the final tree is **20 30 40 50 60 70 80** (sorted), which confirms it is a BST.

Diagram (refer SLM): Fig 3.2.5 Process of inserting 51; Fig 3.2.6 Insertion operation.

3.2.4 Deletion Operation in BST

Theory

Three cases when deleting a node with key `Val`:

Case 1 — Leaf node: Set parent's pointer to NULL; free the node.

Example: delete 49 (Fig 3.2.7).

Case 2 — One child: Replace the node by its only child (parent points to that child); free the node.

Example: delete 48 which has only child 49 (Fig 3.2.8).

Case 3 — Two children (internal): Find **inorder successor**, copy its data into the node, then delete the successor (which has 0 or 1 child — Case 1/2).

Inorder successor: Go to **right child**, then keep going **left** until NULL. Successor = minimum key in right subtree.

```

struct node *findSucc(struct node *ptr) {
    struct node *succ = ptr->right;
    while (succ->left != NULL)
        succ = succ->left;
    return succ;
}

```

Important Points

- 3 deletion cases: leaf / one child / two children
- Two-child case uses inorder successor (leftmost of right subtree)
- After deletion, BST property must still hold
- Smallest key = leftmost node; largest = rightmost node

For Exam

BST deletion has three cases. A leaf is removed by nulling the parent link. A one-child node is replaced by its child. For two children, copy the inorder successor's value into the node and delete the successor. The successor is the leftmost node of the right subtree.

Diagram (refer SLM): Fig 3.2.7 Leaf deletion; Fig 3.2.8 One-child deletion; Fig 3.2.9 Inorder successor; Fig 3.2.10 Internal-node deletion.

Unit Recap (Unit 2 — from SLM)

- BST: left < node < right; unique keys; recursive property
- Inorder → ascending order
- Search / insert follow comparison path
- Delete: leaf, one child, or two children (via inorder successor)

Unit 3: Balancing Binary Tree

3.3.1 Concept of Balancing Tree Data Structure

Theory

If a tree is **skewed**, paths to some leaves are much longer than others — search and message-passing (SLM mobile-tower analogy) become slow. **Balancing** keeps height small so leaves are reached with fewer steps.

- **Perfectly height-balanced:** left and right subtrees of root at same height (rare).
- **Height-balanced:** for every node, $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$ ("almost perfect").

Important Points

- Aim: minimum height → faster search
- Balanced trees support dynamic sorted sets efficiently

- Unbalanced BST (sorted insert order) degenerates to a list

For Exam

Tree balancing keeps the heights of left and right subtrees close so operations stay efficient. A tree is height-balanced if for every node the subtree heights differ by at most one.

Diagram (refer SLM): Fig 3.3.1–3.3.2 Mobile route / tower analogy; Fig 3.3.3 Perfectly balanced; Fig 3.3.4 Height-balanced.

3.3.1.1 Advantages of Balancing

Theory

1. Reach a leaf with **minimum traversals** (better search)
 2. Maintain a dynamic set in sorted order while supporting many operations
-

3.3.2 Balanced Binary Tree

Theory

Search time in a BST depends on **height**. A **Balanced BST** requires:

1. $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$ for every node
2. Left subtree is balanced
3. Right subtree is balanced

Insertions/deletions can destroy balance — hence self-balancing trees (AVL, B-tree).

Important Points

- Ideal height difference ≤ 1 everywhere
- Efficiency of search tied to height

For Exam

A height-balanced binary tree (balanced BST) has, for every node, left and right subtree heights differing by at most one, and both subtrees themselves balanced.

3.3.3 AVL Tree

Theory

AVL tree (Adelson-Velsky & Landis) is a **self-balancing BST**: after every insert/delete, heights of left and right subtrees differ by **at most 1**, while the BST order property is preserved.

Building-block analogy (Fig 3.3.5–3.3.6): structure with height difference 2 is unstable; difference 1 is stable — same idea as AVL.

Two properties to preserve:

1. **Height-balanced:** $|H_L - H_R| \leq 1$
2. **BST property:** left keys < root < right keys

Balanced trees need fewer comparisons in the worst case than skewed trees (SLM comparison of Fig 3.3.7 vs 3.3.8).

- **PYQ Q33 For Exam:** Define AVL; state BF; explain LL/RR (single rotations) and LR/RL (double rotations) with before/after sketches; mention inventors and $O(\log n)$ height.

Important Points

- Named after Georgy Adelson-Velsky and Landis
- Self-balancing BST
- Application example in SLM: telephone/line connection style routing
- Rebalancing via **rotations**

For Exam

An AVL tree is a self-balancing binary search tree in which the heights of the left and right subtrees of every node differ by at most one. It preserves BST order and restores balance after updates using rotations. Named after Adelson-Velsky and Landis.

Previously Asked Questions

- **Q33** (4 marks, Apr 2025) — Explain AVL tree.
 - *Answer:* An AVL tree is a **self-balancing binary search tree** named after Adelson-Velsky and Landis. It follows all BST rules ($\text{left} < \text{node} < \text{right}$), and additionally keeps the tree height-balanced.

For every node, the **balance factor** $BF = \text{height}(\text{left}) - \text{height}(\text{right})$ must be **-1, 0 or +1**. If $|BF|$ becomes 2 or more after insert/delete, the tree is unbalanced and is corrected using **rotations**:

- LL (single right rotation)
- RR (single left rotation)
- LR and RL (double rotations)

Because the tree stays balanced, search, insert and delete remain efficient (about $O(\log n)$ height).

Diagram (refer SLM): Fig 3.3.5–3.3.6 Blocks; Fig 3.3.7–3.3.10 AVL vs non-AVL examples.

3.3.3.1 Balancing Factor

Theory

Balance Factor (BF) of a node:

```
[  
BF = H_L - H_R  
]
```

(height of left subtree – height of right subtree)

For an AVL node, $BF \in \{-1, 0, +1\}$.

$|BF| \geq 2 \rightarrow$ node (and tree) is **unbalanced**.

SLM example (Fig 3.3.11): Leaf → BF 0; node with only left child → BF = 1; root 65 has H_L=2, H_R=1 → BF = 1.

Important Points

- $BF = H_L - H_R$
- Valid AVL BF: -1, 0, 1
- Often written beside each node in diagrams

For Exam

Balance factor of a node is $\text{height}(\text{left}) - \text{height}(\text{right})$. In an AVL tree every node's BF is -1, 0 or 1; if $|BF|$ becomes 2 or more, rotations rebalance the tree.

Diagram (refer SLM): Fig 3.3.11 AVL tree with BF.

3.3.3.2 Unbalanced Trees and Rotations

Theory

Imbalance appears after insert/delete. Four patterns (named by where the extra height appears relative to the unbalanced node):

Case	Name	Cause pattern	Fix
1	LL (Left of Left)	Left-high, then left again	Single right rotation
2	RR (Right of Right)	Right-high, then right again	Single left rotation
3	RL (Right of Left)	Left-high, but left child right-high	Left rotate child, then right rotate root
4	LR (Left of Right)	Right-high, but right child left-high	Right rotate child, then left rotate root

Worked ideas (from SLM):

LL: Insert makes left of left heavy (e.g., insert under 9 below 19). Balance by rotating the unbalanced node **right** so former left child becomes new subtree root (Fig 3.3.16–3.3.17). Relocate orphaned middle subtree to preserve BST order.

RR: Mirror of LL — rotate unbalanced root **left** (Fig 3.3.18).

RL: First rotate the left subtree **left**, then rotate the root **right** (Fig 3.3.19 a–c).

LR: First rotate the right subtree **right**, then rotate the root **left** (Fig 3.3.20 a–c).

Important Points

- Four imbalance cases: LL, RR, RL, LR
- Single rotation for LL/RR; double for RL/LR
- Rotations restore BF while keeping BST order
- Always rebalance at the lowest unbalanced ancestor

For Exam

AVL imbalance is classified as LL, RR, RL or LR. LL is fixed by a right rotation; RR by a left rotation; RL by left-then-right; LR by right-then-left. Rotations rearrange links without breaking the search-tree property.

Diagram (refer SLM): Fig 3.3.12–3.3.15 Unbalanced cases; Fig 3.3.16–3.3.20 Balancing rotations.

3.3.4 B-Tree

Theory

A **B-tree** is a self-balancing multiway search tree: each node may hold **many keys** and have **more than two children**. Designed so node size $\approx$ **disk block**, keeping tree **short (fat)** to reduce expensive disk I/O.

Features (SLM):

- All leaves at the **same level**
- Defined by minimum degree **t** (depends on disk block size)
- Every node except root has at least **t-1** keys; root ≥ 1 key
- Every node has at most **2t-1** keys
- Children of a node = number of keys + 1
- Keys in a node sorted; child between k_1 and k_2 holds keys in (k_1, k_2)
- Grows/shrinks from the **root** (unlike BST which grows downward)
- Search / insert / delete: **$O(\log n)$**

Operations (overview):

- **Search:** like multiway BST — compare within node, descend to correct child
- **Insert:** insert in leaf; if full, **split**, push middle key to parent
- **Delete:** remove from leaf or replace internal key by predecessor/successor; **borrow** or **merge** if underflow

Important Points

- Height-balanced m-way tree for secondary storage
- Fat & short $\rightarrow$ fewer disk accesses than AVL for huge data
- Not all data assumed in RAM (unlike typical AVL/Red-Black discussion)

For Exam

A B-tree is a self-balancing search tree whose nodes store multiple keys and multiple children. All leaves are at the same level. It reduces disk accesses by keeping height low; complexity of main operations is $O(\log n)$. It grows from the root and is suited to large databases on secondary storage.

Diagram (refer SLM): Fig 3.3.21 B-Tree.

Unit Recap (Unit 3 — from SLM)

- Balance $\rightarrow$ shorter paths $\rightarrow$ faster search
- AVL = self-balancing BST; $BF = H_L - H_R \in \{-1, 0, 1\}$

- Fix imbalance with LL/RR/RL/LR rotations
 - B-tree = multi-key, multi-child, disk-oriented balanced tree
-

Unit 4: Graphs

3.4.1 Definition of Graph

Theory

A **graph** G consists of:

- A non-empty finite set of **vertices** $V(G) = \{v_0, v_1, \dots, v_n\}$
- A finite set of **edges** $E(G) = \{e_1, e_2, \dots\}$

Each edge joins a pair of vertices. If $e = (v_i, v_j)$, then v_i and v_j **lie on** e , and e is **incident** with both.

Unlike trees (one parent, many children hierarchy), graphs allow many-to-many relationships. **Every tree is a graph, but not conversely.**

Route-map model: junctions = vertices, roads = edges (Fig 3.4.1).

Example:

$V(G) = \{A, B, C, D, E, F\}$

$E(G) = \{(A, B), (A, D), (B, C), (D, C), (C, F), (D, E), (D, F)\}$ — 6 vertices, 7 edges.

- **PYQ Q7 For Exam:** A graph G is a non-linear structure with a non-empty finite vertex set $V(G)$ and a finite edge set $E(G)$, where each edge connects a pair of vertices.

Important Points

- Graph = (V, E)
- Models pairwise relationships (maps, circuits, networks)
- Tree $\subset$ Graph

For Exam

A graph G consists of a non-empty finite set of vertices $V(G)$ and a finite set of edges $E(G)$. An edge joins two vertices (endpoints). Graphs model networks such as roads, flights and social links.

Previously Asked Questions

- **Q7** (1 mark, Apr 2025) — Define the term “graph.”
 - *Answer:* A graph is a non-linear data structure consisting of a set of vertices (nodes) and a set of edges that connect pairs of vertices; denoted $G = (V, E)$.

Diagram (refer SLM): Fig 3.4.1 Route map → graph; Fig 3.4.2 Simple graph; Fig 3.4.3 Problem graph with 5 vertices.

3.4.2 Types of Graphs (Directed / Undirected)

Theory

Undirected graph: Edge endpoints are an **unordered** pair — (v_1, v_2) same as (v_2, v_1) . Drawn without arrowheads.

Directed graph (digraph): Edge is an **ordered** pair (v_1, v_2) — $v_1 = \text{tail}$, $v_2 = \text{head}$. Drawn with arrowheads. $(A, D) \neq (D, A)$.

Important Points

- Two broad types: undirected & directed
- Undirected: bidirectional meaning
- Directed: one-way arcs

For Exam

Graphs are broadly undirected or directed. In an undirected graph edges are unordered pairs; in a digraph edges are ordered pairs with a direction from tail to head.

Diagram (refer SLM): Fig 3.4.4 Undirected; Fig 3.4.5 Directed.

3.4.3 Terminologies Used in Graphs

3.4.3.1 Weighted Graph

Edges labelled with numbers (distance, cost, speed limit) → **weighted graph**.

Diagram (refer SLM): Fig 3.4.6 Weighted graph (weights 4,2,5,3).

3.4.3.2 Self Loop

Edge from a vertex to **itself** (v, v) .

Diagram (refer SLM): Fig 3.4.7 Self loop.

3.4.3.3 Parallel Edges

More than one edge between the **same** pair of vertices.

Diagram (refer SLM): Fig 3.4.8 Parallel edges.

3.4.3.4–3.4.3.5 Adjacent Vertices & Incidence

- **Adjacent:** edge exists between u and v
- **Incidence:** undirected edge (u, v) incident on u and v ; directed (u, v) incident **from** u **to** v

Diagram (refer SLM): Fig 3.4.9 Adjacent / incidence.

3.4.3.6 Degree of Vertex

- **Undirected:** degree = number of incident edges
- **Directed:** **in-degree** (edges in) + **out-degree** (edges out)

Diagram (refer SLM): Fig 3.4.10 Undirected degrees; Fig 3.4.11 Directed in/out degrees.

3.4.3.7 Simple Graph

No self-loops and no parallel edges.

Diagram (refer SLM): Fig 3.4.12 Simple graphs.

3.4.3.8 Multi Graph

A graph with a **self-loop or parallel edges or both** is a **multi-graph**.

- **PYQ Q13 For Exam: Multi-graph** (or multigraph).

Previously Asked Questions

- **Q13** (1 mark, Apr 2025) — What is the name of a graph which has either a self-loop or parallel edges or both?
 - *Answer:* Multi-graph (or multigraph).

Diagram (refer SLM): Fig 3.4.13 Multi-graph.

3.4.3.9 Maximum Edges

- Simple undirected: max edges = $n(n-1)/2$
- Simple directed: max edges = $n(n-1)$

Diagram (refer SLM): Fig 3.4.14 Maximum edges.

3.4.3.10 Complete Graph

Every vertex adjacent to every other vertex. Undirected complete graph has $n(n-1)/2$ edges.

Diagram (refer SLM): Fig 3.4.15 Complete graph ($n=4 \rightarrow 6$ edges).

3.4.3.11 Regular Graph

Every vertex has the **same degree**.

Diagram (refer SLM): Fig 3.4.16 Regular graph (all degree 3).

3.4.3.12 Planar Graph

Can be drawn in a plane with **no edge crossings**.

Diagram (refer SLM): Fig 3.4.17 Planar graphs.

3.4.3.13 Walk & Path

- **Walk:** finite sequence of adjacent edges; length = number of edges
- **Trail:** walk with distinct edges
- **Path:** trail with distinct vertices

- Closed path/trail if start = end

Diagram (refer SLM): Fig 3.4.18 Walk and path examples.

3.4.3.14–3.4.3.16 Cycle; Cyclic & Acyclic

- **Cycle:** closed path with ≥ 1 edge
- **Cyclic graph:** contains a cycle
- **Acyclic graph:** no cycles

Diagram (refer SLM): Fig 3.4.19 Cycle; Fig 3.4.20 Cyclic; Fig 3.4.21 Acyclic.

3.4.3.17 Connected Graph

- Undirected **connected:** path between every pair of vertices
- Digraph **strongly connected:** directed path $u \rightarrow v$ for every ordered pair
- Digraph **weakly connected:** underlying undirected graph is connected

Diagram (refer SLM): Fig 3.4.22 Connected / disconnected / strong / weak.

3.4.3.18–3.4.3.20 Articulation Point, Bridge, Biconnected

- **Articulation point:** removal disconnects the graph
- **Bridge:** edge whose removal disconnects the graph
- **Biconnected:** no articulation points

Diagram (refer SLM): Fig 3.4.23 Articulation; Fig 3.4.24 Bridge; Fig 3.4.25 Biconnected.

Important Points (terminology pack)

- Weighted / simple / multi / complete / regular / planar
- Walk $\supset$ trail $\supset$ path; cycle = closed path
- Connected vs disconnected; strong vs weak (digraphs)
- Articulation point & bridge; biconnected = no articulation points

For Exam

Graph terminology includes weighted edges, loops, parallel edges, degree (in/out), simple vs multi-graph, complete and regular graphs, walks/paths/cycles, connectedness, articulation points and bridges. A multi-graph has loops and/or parallel edges; a simple graph has neither.

3.4.4 Graph Representation — Adjacency Matrix

Theory

Adjacency matrix A is an $n \times n$ matrix ($n = |V|$):

- $a_{ij} = 1$ if edge from V_i to V_j exists; else 0 (Bit/Boolean matrix)
- Undirected $\rightarrow$ matrix **symmetric** ($A = A^T$)

- Undirected: degree of i = **row sum**
- Directed: **out-degree** = row sum; **in-degree** = column sum
- Space: n^2 bits
- Multi-graph: store **count** of edges; weighted: store **weights**

Worked example (digraph Fig 3.4.27): edges $1 \rightarrow 2$, $1 \rightarrow 4$, $2 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 4$, $4 \rightarrow 1$ give matrix rows:

`[0 1 0 1] / [0 0 1 1] / [0 0 0 1] / [1 0 0 0]`

Important Points

- Sequential representation
- Fast edge-existence test $O(1)$
- Costly for sparse graphs (many zeros); awkward insert/delete of vertices

For Exam

An adjacency matrix is an $n \times n$ Boolean (or weight) matrix with $a_{ij} = 1$ if an edge exists from i to j . Undirected graphs give symmetric matrices. Space is $O(n^2)$. Row/column sums give degrees.

Diagram (refer SLM): Fig 3.4.26–3.4.30 Matrix examples.

3.4.5 Adjacency List Representation

Theory

Adjacency list = array of **linked lists** (one list per vertex). List for v stores neighbours of v . Better when n is large / graph is sparse; easier insert/delete of vertices than resizing a matrix.

Undirected (Fig 3.4.31): each edge appears in **both** endpoints' lists.

Directed (Fig 3.4.32): list stores only **out-neighbours**.

Multi-list representation: edge-centric — directory of vertex headers + edge nodes with mark bit, two endpoints, and two list links (Fig 3.4.33–3.4.35).

Important Points

- Linked (non-sequential) representation
- Space $\approx O(n + e)$
- Array index = vertex; nodes = adjacent vertices

For Exam

Adjacency list representation stores for each vertex a linked list of adjacent vertices. Undirected edges appear twice; digraphs store outgoing neighbours only. It saves space for sparse graphs compared with an $n \times n$ matrix.

Diagram (refer SLM): Fig 3.4.31 Undirected list; Fig 3.4.32 Directed list; Fig 3.4.33–3.4.35 Multi-list.

3.4.6 Implementation of Graph (C sketch)

Theory

Non-weighted adjacency-list node:

```
#define MAX25
typedef struct node {
    int vertex;
    struct node *next;
} node1;
node1 *adj[MAX];
```

Weighted: add `int weight;` field (`node2`).

Important Points

- `adj[i]` heads the neighbour list of vertex `i`
- Typedef + self-referential struct = classic linked-list pattern

3.4.7 Graph Traversal

Theory

Graph traversal finds vertices reachable from a start set and orders visits **without looping forever**. Differences from tree traversal:

- No unique root — start at **any** vertex
- May need several starts to cover a disconnected graph
- Mark visited vertices to avoid revisiting

Two main methods:

	BFS	DFS
Structure	Queue	Stack
Strategy	Level / breadth	Dive deep + backtrack
Nature (SLM note)	Vertex-based	Edge-based
Prefer when target is...	Near start	Far from start

3.4.7.1 Breadth First Search (BFS)

Theory

Visit a vertex, then **all its neighbours**, then their neighbours, and so on — using a **queue**.

Algorithm:

1. Create empty queue
2. Enqueue start vertex (mark visited)
3. While queue not empty: dequeue front `u`; enqueue all unvisited neighbours of `u`
4. Stop when queue empty

Sample in C (simple outline)

```
/* adj[][] = adjacency matrix; visited[] starts as 0 */
void BFS(int start, int n) {
    int queue[20], front = 0, rear = -1, u, v;
    visited[start] = 1;
    queue[++rear] = start;
    while (front <= rear) {
        u = queue[front++];
        printf("%d ", u);          /* visit */
        for (v = 0; v < n; v++)
            if (adj[u][v] && !visited[v]) {
                visited[v] = 1;
                queue[++rear] = v;
            }
    }
}
```

Worked example (Fig 3.4.36, start A):

Neighbours of A: B, C, D ...

1. Queue: [A]
2. Visit A → enqueue B,C,D → order so far A
3. Visit B → enqueue E (D already queued)
4. Visit C → D already present
5. Visit D → E already present
6. Visit E → done

BFS order: A, B, C, D, E

Important Points

- Queue FIFO = explore by distance from start
- Good for shortest path in **unweighted** graphs
- Mark visited to avoid cycles

For Exam

BFS traverses a graph level by level using a queue: enqueue the start, then repeatedly dequeue a vertex and enqueue its unvisited adjacent vertices until the queue is empty. Example from A: order A B C D E.

Diagram (refer SLM): Fig 3.4.36–3.4.42 BFS steps with queue.

3.4.7.2 Depth First Search (DFS)

Theory

From the start, go as **deep** as possible along one path, then **backtrack**, using a **stack**.

Algorithm:

1. Create empty stack
2. Push start
3. While stack not empty: visit an unvisited adjacent of the **top**; push it; if none, **pop** (backtrack)
4. Repeat until stack empty

Sample in C (recursive DFS)

```
void DFS(int u, int n, int visited[]) {
    int v;
    visited[u] = 1;
    printf("%d ", u);
    for (v = 0; v < n; v++) {
        if (adj[u][v] == 1 && visited[v] == 0)
            DFS(v, n, visited);
    }
}
```

Worked example (Fig 3.4.43, start A):

1. Push A
2. Visit B → push B
3. Visit D → push D
4. Visit E → push E
5. E has no new adj → pop E
6. D done → pop D
7. B done → pop B
8. From A visit remaining neighbour C → push C
9. C done → pop C
10. A done → pop A; stack empty

One valid DFS visit order along this path choice: **A, B, D, E, C** (then finish).

- **PYQ Q19 For Exam:** DFS = depth-oriented graph traversal using a stack and backtracking. Start at a source, repeatedly move to an unvisited adjacent vertex; if none, pop and backtrack until all reachable vertices are visited. Give the stack walkthrough for a small graph (as above).

Important Points

- Stack + backtracking
- Explores one branch fully before siblings

- Useful for connectivity, topological ideas, maze/path existence

For Exam

Depth-first search starts at a source vertex and explores as far as possible along each branch before backtracking. It uses a stack: push the start, push an unvisited neighbour of the top, and pop when the top has no new neighbours, until the stack is empty. Example starting at A may visit A-B-D-E then backtrack and visit C.

Previously Asked Questions

- **Q19** (2 marks, Apr 2025) — Define depth-first search (DFS) with an example.
 - *Answer:* Depth-first search (DFS) is a graph traversal method that starts at a source vertex and explores as **deep** as possible along one path before **backtracking**. It uses a **stack**. Push the start vertex; then repeatedly push an unvisited adjacent vertex of the top; if the top has no new neighbour, pop it (backtrack). Continue until the stack is empty.
 - Example:** Starting from A, one possible visit order is **A, B, D, E, C** (go deep $A \rightarrow B \rightarrow D \rightarrow E$, then backtrack and visit C).

Diagram (refer SLM): Fig 3.4.43–3.4.53 DFS steps with stack.

3.4.8 Applications of Graph

Theory

Graphs model relationships between entities. Major SLM applications:

3.4.8.1 Traveling Salesman Problem (TSP)

Visit every city exactly once and return home on a **minimum-length** tour. Cities = vertices; roads = weighted edges. Compare tours (e.g., A-B-C-D-E-A length 24 vs A-B-C-E-D-A length 31) and choose the shortest.

Diagram (refer SLM): Fig 3.4.54 TSP.

3.4.8.2 Google Maps / GPS / Flight Networks

- **Maps:** locations & junctions = vertices; roads = weighted edges; shortest-path algorithms recommend routes (e.g., Wadakkanchery → Kunnamkulam).
- **GPS:** often single-source distances; BFS can list neighbourhoods; devices may store maps offline.
- **Flights:** airports = vertices; flights = directed edges; used for route and fuel optimisation.

Diagram (refer SLM): Fig 3.4.55–3.4.58 Maps / GPS / flights.

3.4.8.3 Social Networks

Users/pages/photos = vertices; friendships/likes/follows = edges. Facebook friendship ≈ **undirected**; Twitter follows ≈ **directed**. Friend suggestion ≈ find nodes at shortest-path

distance 2.

Diagram (refer SLM): Fig 3.4.59 Social network.

3.4.8.4 E-Commerce Recommendations

Users and products = vertices; purchase/interest = edges. Recommend items bought by users who share a product with you (common-neighbour style on the bipartite user-product graph).

Diagram (refer SLM): Fig 3.4.60–3.4.61 Recommendation graphs.

3.4.9 Knowledge Graphs

Theory

A **knowledge graph** is a knowledge base using a graph data model: entities/events/concepts = **nodes**; relationships = **labelled edges** (e.g., "Director of", "located in", "painted"). Integrates facts into an interlinked structure (travel diary example; Spielberg filmography example).

Diagram (refer SLM): Fig 3.4.62–3.4.63 Knowledge graphs.

3.4.8–3.4.9 Combined Exam Focus (Types, Representations, Applications)

- **PYQ Q38 For Exam structure (15 marks):**

1. **Definition** of graph
2. **Types:** undirected/directed; then weighted, simple, multi, complete, regular, planar, cyclic/acyclic, connected (strong/weak)
3. **Representations:** adjacency matrix (with example) + adjacency list (with example); brief multi-list
4. **Applications:** TSP, maps/GPS/flights, social networks, e-commerce recommendations, knowledge graphs
5. Mention BFS/DFS as traversal tools used in applications

Previously Asked Questions

- **Q38** (15 marks, Apr 2025) — Discuss the types of graphs, their representations, and applications in computing.
 - *Answer:*
(Expand each point with one example diagram/table from the notes for a full 15-mark answer.)
 - **Definition:** A graph $G = (V, E)$ is a non-linear structure of vertices V and edges E connecting pairs of vertices.
 - **Types:** Undirected and directed (digraph); weighted and unweighted; simple graph (no loop/parallel edges); multi-graph (self-loop and/or parallel edges); complete graph; connected / disconnected; cyclic / acyclic; regular; planar, etc.
 - **Representations:**

- **Adjacency matrix:** $n \times n$ matrix; entry $(i,j) = 1$ (or weight) if edge exists, else 0. Easy to check an edge; uses $O(n^2)$ space.
- **Adjacency list:** For each vertex, store a linked list of its neighbours. Better for sparse graphs; uses less space.
- **Applications:** Travelling Salesman Problem; Google Maps / GPS shortest path; flight networks; social networks (friends as edges); e-commerce recommendations; knowledge graphs. Traversal methods BFS (queue) and DFS (stack) are used in many of these applications.

For Exam (essay outline)

A graph $G=(V,E)$ models pairwise relationships. Types include undirected and directed graphs, and further classes such as weighted, simple, multi-graph, complete, regular, planar, cyclic/acyclic and connected graphs. Representations: adjacency matrix ($n \times n$, symmetric if undirected) and adjacency lists (array of neighbour lists; better for sparse graphs). Applications span TSP, navigation (Google Maps/GPS), flight networks, social networks, e-commerce recommendations and knowledge graphs — all relying on vertices, edges and often shortest-path or traversal algorithms (BFS/DFS).

Unit Recap (Unit 4 — from SLM)

- Graph = vertices + edges; tree is a special graph
 - Undirected vs directed; rich terminology (degree, path, cycle, multi-graph, ...)
 - Store as adjacency matrix or adjacency list (or multi-list)
 - Traverse with BFS (queue) or DFS (stack)
 - Applications: TSP, maps, social nets, recommendations, knowledge graphs
-

Block 4: Complexity of Algorithms

Unit 1: Complexity of Algorithms

4.1.1 Essential Properties of Algorithms

Theory

An algorithm is a finite set of unambiguous, step-by-step instructions that solves a given problem and terminates after a finite number of steps. Just as there can be many ways to prepare a pizza, there can be many algorithms for the same computational problem. We therefore study algorithms not only for correctness, but also to compare which one saves time and memory.

Five essential properties make a procedure an algorithm: it may take inputs, must produce at least one output, every step must be definite (clear), it must be finite (must stop), and every instruction must be effective (feasible and useful).

Important Points

- **Algorithm** = finite, unambiguous steps to solve a problem

- **1. Input** — zero or more data items supplied before execution
- **2. Output** — at least one result produced after execution
- **3. Definiteness** — every instruction is clear and unambiguous
- **4. Finiteness** — algorithm terminates after a finite number of steps
- **5. Effectiveness** — every instruction is feasible and does some work
- Different algorithms may solve the same problem; we choose the efficient one

For Exam

An algorithm is a finite set of step-by-step unambiguous instructions that accomplish a particular task. Its essential properties are: input (zero or more initial data items), output (at least one result), definiteness (each step is clear), finiteness (it terminates after finite steps), and effectiveness (each instruction is feasible and performs useful work).

Previously Asked Questions

(None mapped specifically to properties alone; related complexity PYQs appear under 4.1.3 / 4.1.7.)

4.1.2 Cases to Consider During Analysis

Theory

The performance of an algorithm depends heavily on the kind of input provided. For example, if a list is almost sorted, some sorting algorithms run quickly while others do not; if the list is random, the opposite may be true. Therefore, while analysing an algorithm we must consider multiple input sets: best-case, worst-case, and average-case inputs.

Important Points

- Algorithm performance depends on the **nature of input**
- Three cases during analysis: **Best, Worst, Average**
- Same algorithm can behave differently on sorted vs reverse-sorted vs random data
- Analysis compares algorithms by time (operations) and/or space (memory)

For Exam

While analysing an algorithm, three input cases are considered because performance varies with input: best-case input (fastest run), worst-case input (slowest run), and average-case input (typical/expected performance). Multiple input sets must be studied because a nearly sorted list and a randomly arranged list can produce very different running times for the same sorting algorithm.

4.1.2.1 Best Case Input

Theory

Best-case input is the input set that allows the algorithm to do the least work and finish in the shortest time. Example: searching a list when the required item is the first element — only one

comparison is needed. In the pizza analogy, this is like finding salt immediately because ingredients are already arranged orderly.

Important Points

- Best case = input that makes the algorithm run **quickest**
- Causes the algorithm to do the **least amount of work**
- Search example: key found at the **first** position → minimum comparisons
- Often expressed using **Big-Omega (Ω)** as a lower bound (SLM association)

For Exam

Best-case input is the input set for which an algorithm performs the least amount of work and takes the shortest time. For sequential search, finding the target as the first element is a best-case situation.

4.1.2.2 Worst Case Input

Theory

Worst-case input is the input set that forces the algorithm to do the maximum amount of work and take the longest time. Example: searching when the key is the last element, or is absent — the whole list must be examined. This case is especially important in practice because it gives a guaranteed upper bound on running time.

Important Points

- Worst case = input that makes the algorithm run **slowest**
- Causes **maximum work** / maximum time
- Search example: key at **last** position (or not present) → n comparisons
- Usually expressed using **Big-O (O)** as an upper bound

For Exam

Worst-case input is the input that makes an algorithm perform at its slowest by forcing maximum work. In searching a list of n elements, finding the key only at the end (or not finding it) is a typical worst case.

4.1.2.3 Average Case Input

Theory

Average-case input represents a typical or “expected” situation — neither the most favourable nor the most hostile arrangement. Average-case complexity is often computed by considering all possible inputs (or a realistic probability model) and taking the mean number of operations. For many algorithms, average-case order matches the worst-case order.

Important Points

- Average case = **typical / expected** performance over inputs
- Not necessarily equal to “halfway between best and worst,” but often close in order

- For insertion sort, SLM notes average is often same order as worst case: **$O(n^2)$**
- Often associated with **Big-Theta (Θ)** tight bound in SLM presentation

For Exam

Average-case input is the input set that yields average (typical) performance of an algorithm. It reflects expected behaviour for common inputs and is used along with best and worst cases when comparing algorithms.

4.1.3 Complexity of Algorithms

Theory

To decide which algorithm is better, we need measurable criteria. The two main measures of efficiency are **time complexity** (how running time grows with input size) and **space complexity** (how memory requirement grows with input size). Complexity analysis focuses on growth rate for large n , not on absolute seconds on a particular machine.

Important Points

- Efficiency measures: **Time complexity** and **Space complexity**
- Helps compare alternative algorithms for the same problem
- Time complexity $\approx$ growth of noticeable operations as function of input size n
- Usually expressed using **asymptotic notations** (O , Ω , Θ)

For Exam

Complexity of an algorithm measures its efficiency. Time complexity is the time taken as a function of input length; space complexity is the memory required as a function of input characteristics. These measures allow comparison of algorithms independently of a specific computer's speed.

4.1.3.1 Time Complexity

Theory

Time complexity is the amount of time taken by an algorithm to run as a function of the length of the input. It estimates how the number of basic operations grows when n becomes large. In cooking terms: if ingredients are ready and the method is simple, time is minimal; if organisation is poor and the method is complex, time increases sharply. In algorithms, we count meaningful operations rather than wall-clock seconds.

Important Points

- Time complexity = time (operations) as a function of input size n
- Measures growth rate of algorithm $T(n)$ for large n
- Written using asymptotic notation (commonly Big-O)
- Constants and lower-order terms are ignored for large n

- **PYQ Q22 answer:** Time complexity is the amount of time taken by an algorithm to run as a function of the length of the input / a theoretical estimate of the growth rate of $T(n)$ for large n

For Exam

Time complexity of an algorithm is the amount of time it takes to run as a function of the length of the input. It is a theoretical estimate of how the running time $T(n)$ grows when the input size n becomes large, and is usually expressed using asymptotic notations such as Big-O.

Previously Asked Questions

- **Q22** (2 marks, Apr 2025) — Define the time complexity of an algorithm.
 - *Answer:* Time complexity is the amount of time taken by an algorithm to run as a function of the length of the input. It estimates how the number of basic operations $T(n)$ grows when input size n becomes large, usually expressed using Big-O notation.

4.1.3.2 Space Complexity

Theory

Space complexity is the amount of memory space required to solve an instance of a problem as a function of the characteristics of the input. It includes memory for variables, data structures, and (for recursive algorithms) the call stack, until the algorithm finishes.

Important Points

- Space complexity = memory needed until the algorithm completes
- Depends on input size and auxiliary structures used
- In-place algorithms (e.g., selection sort, insertion sort) often need **$O(1)$** extra space
- Recursive divide-and-conquer methods may need extra stack space

For Exam

Space complexity of an algorithm is the amount of memory space required to solve a problem instance as a function of the input. It accounts for all memory used until execution completes.

4.1.4 Estimating Complexity / 4.1.4.1 Time for an Algorithm to Run $T(n)$

Theory

Let $T(n)$ denote the time (number of operations) for input size n . To estimate complexity we study how $T(n)$ grows. The SLM illustrates this with **insertion sort** on `[9, 4, 6, 2, 5, 3]`: the key is compared with previous elements and larger elements are shifted right until the key's correct place is found.

- **Best case (already sorted):** only $n-1$ comparisons, no (or minimal) shifts → **$O(n)$**
- **Worst case (reverse sorted):** about $1+2+\dots+(n-1)$ comparisons and the same order of shifts → **$O(n^2)$**
- **Average case:** typically same order as worst → **$O(n^2)$**

Important Points

- $T(n)$ = running-time function of input size n
- Insertion sort best case (sorted array): **$O(n)$**
- Insertion sort worst/average (reverse / random): **$O(n^2)$**
- Operations counted: comparisons + shifts/swaps
- Worst-case scans/swaps $\approx 2 \times (1+2+\dots+n-1) = n(n-1) \rightarrow O(n^2)$
- Focus on dominant term for large n

Worked sketch (insertion sort idea from SLM)

Array: 9 4 6 2 5 3

1. Key=4 $\rightarrow$ shift 9 $\rightarrow$ 4 9 6 2 5 3
2. Key=6 $\rightarrow$ shift 9 $\rightarrow$ 4 6 9 2 5 3
3. Key=2 $\rightarrow$ shift 9,6,4 $\rightarrow$ 2 4 6 9 5 3
4. Key=5 $\rightarrow$ shift 9,6 $\rightarrow$ 2 4 5 6 9 3
5. Key=3 $\rightarrow$ shift 9,6,5,4 $\rightarrow$ 2 3 4 5 6 9

For Exam

$T(n)$ measures how long an algorithm takes for input size n . For insertion sort, the best case occurs when the array is already sorted and $T(n)=O(n)$; the worst case occurs for a decreasing list and needs about $n(n-1)$ operations, giving $T(n)=O(n^2)$. Average case is also $O(n^2)$.

4.1.5 Reasons to Analyse Algorithms

Theory

A problem may have many correct algorithms. Analysis tells us which is more efficient by comparing time complexity and/or space complexity. Time complexity counts the noticeable operations and studies the growth of $T(n)$ for large n , usually via asymptotic notations.

Important Points

- Multiple algorithms may solve one problem — analysis chooses the better one
- Compare by **time** and/or **space**
- Time complexity = theoretical growth rate of $T(n)$ for large n
- Expressed with asymptotic notations

For Exam

We analyse algorithms to determine which of several correct solutions is more efficient. Analysis compares the time required (time complexity) and/or memory required (space complexity), focusing on how $T(n)$ grows for large input size n .

4.1.6 Asymptotic Notations

Theory

Asymptotic notations describe how an algorithm's running time behaves as input size grows large. They ignore machine-dependent constants and focus on order of growth. The three main notations used in the SLM are:

1. **Big-O (O)** — asymptotic **upper** bound (worst-case oriented)
2. **Big-Omega (Ω)** — asymptotic **lower** bound (best-case oriented)
3. **Big-Theta (Θ)** — asymptotic **tight** bound (average / matching upper & lower)

An algorithm is considered more efficient if its worst-case running time has a **lower order of growth**.

Important Points

Notation	Bound	SLM association	Meaning
O (Big-Oh)	Upper	Worst case	$f(n)$ grows no faster than $g(n)$
Ω (Big-Omega)	Lower	Best case	$f(n)$ grows at least as fast as $g(n)$
Θ (Big-Theta)	Tight	Average / both	$f(n)$ grows at the same rate as $g(n)$

- Used to communicate complexity unambiguously
- Compare growth for **large n**
- Drop constants and non-dominant terms (e.g., $3n+2 = O(n)$)

Diagram (refer SLM): Fig. 4.1.1 — graphical comparison of O, Ω , Θ bounds around $f(n)$.

For Exam

Asymptotic notations analyse algorithm running time as input size increases. Big-O gives an upper bound (worst-case), Big-Omega a lower bound (best-case), and Big-Theta a tight bound (same order from above and below). They classify growth rates so we can compare algorithms for large n .

4.1.6.1 Big Oh (O) Notation — Asymptotic Upper Bound (Worst Case)

Theory

$f(n)$ is **$O(g(n))$** if there exist constants $c > 0$ and n_0 such that

$f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Example: $f(n)=3n+2$, $g(n)=n$.

For $n \geq 2$, $3n+2 \leq 3n+n = 4n$, so $c=4$, $n_0=2 \rightarrow 3n+2 = O(n)$.

Important Points

- Big-O = **worst-case / upper-bound** complexity (SLM framing)
- Definition: $\exists c > 0, n_0: f(n) \leq c \cdot g(n) \forall n \geq n_0$
- Example: $3n+2 = O(n)$
- Dominating term decides the order (linear term dominates constant)

For Exam

Big-O notation gives an asymptotic upper bound on running time. $f(n)$ is $O(g(n))$ if $f(n) \leq c \cdot g(n)$ for some positive constant c and all sufficiently large n . Thus $3n+2 = O(n)$, meaning runtime grows at most linearly for large n .

4.1.6.2 Big-Omega (Ω) Notation — Asymptotic Lower Bound (Best Case)

Theory

$f(n)$ is $\Omega(g(n))$ if there exist $c > 0$ and $n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n > n_0$.

Example: $3n+2 \geq 3n$ for $n \geq 1$ with $c=3 \rightarrow 3n+2 = \Omega(n)$.

Important Points

- Big- Ω = **best-case / lower-bound** complexity (SLM framing)
- Definition: $\exists c > 0, n_0: f(n) \geq c \cdot g(n)$ for large n
- Example: $3n+2 = \Omega(n)$
- Says runtime grows **at least** as fast as $g(n)$

For Exam

Big-Omega notation provides an asymptotic lower bound. $f(n)$ is $\Omega(g(n))$ if $f(n) \geq c \cdot g(n)$ for some $c > 0$ and all large n . For example, $3n+2 = \Omega(n)$, so the runtime grows at least linearly.

4.1.6.3 Big-Theta (Θ) Notation — Asymptotic Tight Bound (Average Case)

Theory

$f(n)$ is $\Theta(g(n))$ if there exist $c_1 > 0$, $c_2 > 0$, and $n_0 > 0$ such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.

If f is $\Theta(g)$, then it is both $O(g)$ and $\Omega(g)$. Example: $3n+2 = \Theta(n)$ (e.g., $c_1=1$, $c_2=4$, $n_0=2$).

Important Points

- Big- Θ = **tight bound** (upper and lower together)
- SLM links it to **average-case** discussion
- $f = \Theta(g) \Rightarrow f = O(g)$ and $f = \Omega(g)$
- Example: $3n+2 = \Theta(n)$

Comparison table (SLM Table 4.1.1)

Big-O	Big- Ω	Big- Θ
Worst case	Best case	Average / tight
Longest time bound	Shortest time bound	Same-order growth

For Exam

Big-Theta notation gives a tight asymptotic bound: $f(n)$ is $\Theta(g(n))$ if it is sandwiched between two positive constant multiples of $g(n)$ for large n . Thus $3n+2 = \Theta(n)$ means the runtime grows linearly, neither faster nor slower in order.

4.1.7 Analysing Algorithms

Theory

Basic program structures are **sequence**, **selection**, and **iteration**. For complexity, we analyse four building blocks: simple statement, sequence structure, loop structure, and if-then-else structure. Each statement is assumed to take unit time unless it contains loops or calls.

Important Points

- Sequence → order of steps matters
- Selection → some steps skipped by condition
- Iteration → steps repeated until condition met
- Complexity of whole algorithm built from these patterns

For Exam

Analysing algorithms means estimating $T(n)$ from their control structures. Simple statements and fixed sequences are $O(1)$; loops contribute factors of n ; nested loops often give $O(n^2)$; if-then-else takes the maximum of the branches' complexities.

4.1.7.1 Simple Statement

Theory

A single simple statement takes unit time: $T(n)=1$. Since $1 \leq 1 \cdot 1$, we get **$T(n)=O(1)$** — constant time, independent of input size.

Example from SLM: $T(n)=254$. Then $254 \leq 254 \cdot 1$, so $c=254$, $g(n)=1 \rightarrow \mathbf{O(1)}$.

Important Points

- One simple statement → $T(n)=1 \rightarrow \mathbf{O(1)}$
- Any constant $T(n)=k$ (independent of n) → **$O(1)$**
- Constant factors do not change Big-O class

For Exam

A simple statement takes constant time. If $T(n)$ is any constant (e.g., 1 or 254), then $T(n)=O(1)$.

4.1.7.2 Sequence Structure

Theory

Execution time of a sequence is the **sum** of times of its statements. Example: area of rectangle (read length & breadth, multiply, print, stop) — four steps → $T(n)=4 \rightarrow$ still **$O(1)$** because the count does not grow with n .

Important Points

- Sequence time = sum of individual statement times
- Fixed number of statements $\rightarrow O(1)$
- Area-rectangle algorithm: $T(n)=4 = O(1)$

For Exam

In a sequence structure, total time is the sum of the times of the statements. A fixed-length sequence of simple statements has time complexity $O(1)$.

4.1.7.3 The Loop Structure

Theory

Loops make complexity depend on n . Example: counting ones in N inputs — about 4 simple steps outside + 3 operations inside a loop of $N \rightarrow T(n)=3N+4$. For $N \geq 4$, $3N+4 \leq 4N \rightarrow O(N)$.

Nested loops: if both I and J run from 1 to N , statement S executes N^2 times $\rightarrow O(N^2)$.

Important Points

- Single loop $1 \dots N$ with constant body $\rightarrow O(N)$
- Nested loops both $1 \dots N \rightarrow O(N^2)$
- Ignore lower-order terms and constants ($3N+4 \rightarrow O(N)$)
- Worked SLM-style results:
 - $T(n)=2834 \rightarrow O(1)$
 - $T(n)=9n+18 \rightarrow O(n)$
 - $T(n)=18n^2+7 \rightarrow O(n^2)$
 - $T(n)=8n^3+3n^2+6n \rightarrow O(n^3)$

PYQ Q35 — Compute time complexity (Apr 2025)

Expression	Dominant term	Answer	Why
i. $T(n)=410$	constant	$O(1)$	Independent of n ; any constant is $O(1)$
ii. $T(n)=4n-12$	$4n$	$O(n)$	Linear term dominates; for large n , $4n-12 \leq 4n \rightarrow O(n)$
iii. $T(n)=13n^5+4n-7$	$13n^5$	$O(n^5)$	Highest power is n^5 ; lower terms $4n$ and 7 are negligible
iv. $T(n)=3n^2+n+4$	$3n^2$	$O(n^2)$	Quadratic term dominates linear and constant terms

For Exam

A loop that repeats a constant amount of work n times has complexity $O(n)$. Two nested loops each running n times give $O(n^2)$. In a polynomial $T(n)$, Big-O is determined by the highest-degree term: constants are $O(1)$, $an+b$ is $O(n)$, $an^2+\dots$ is $O(n^2)$, $an^5+\dots$ is $O(n^5)$.

Previously Asked Questions

- **Q35** (4 marks, Apr 2025) — Compute the time complexity of: i. $T(n)=410$ ii. $T(n)=4*n-12$ iii. $T(n)=13*n^5+4*n-7$ iv. $T(n)=3*n^2+n+4$
 - *Answer:* In Big-O we keep the **dominant (highest growth)** term and drop constants/lower terms.

- 1. $T(n) = 410 \rightarrow$ constant, independent of $n \rightarrow \mathbf{O(1)}$
- 1. $T(n) = 4n - 12 \rightarrow$ linear term dominates $\rightarrow \mathbf{O(n)}$
- 1. $T(n) = 13n^5 + 4n - 7 \rightarrow$ highest power is $n^5 \rightarrow \mathbf{O(n^5)}$
- 1. $T(n) = 3n^2 + n + 4 \rightarrow$ quadratic term dominates $\rightarrow \mathbf{O(n^2)}$

4.1.7.4 If-then-else Structure

Theory

For `if ... then S1 else S2`, analyse $T1$ and $T2$ separately; the structure's complexity is **max(T1, T2)** (worst branch that may execute).

Example: then-part $O(1)$, else-part a loop $O(N)$ $\rightarrow$ overall **$O(N)$** .

Important Points

- If-then-else complexity = **maximum** of then and else complexities
- Condition test itself is usually $O(1)$
- Always consider the costlier branch for Big-O upper bound

For Exam

In an if-then-else structure, time complexity is the maximum of the complexities of the then-part and the else-part. If one branch is $O(1)$ and the other is $O(n)$, the whole structure is $O(n)$.

Algorithm Complexity — Detailed Exam Note (covers Q39 theory part)

Theory

Algorithm complexity studies how resource usage — mainly **time** and **space** — grows with input size n . Time complexity $T(n)$ counts basic operations; space complexity counts memory. Because absolute time depends on hardware, we use **asymptotic analysis** with Big-O, Big-Ω and Big-Θ to describe best, worst and average behaviour for large n .

Best case uses the most favourable input (least work), worst case the least favourable (most work), and average case a typical mix of inputs. Practical algorithm choice often emphasises **worst-case Big-O**, because it gives a guarantee.

Important Points — Full picture for 15-mark answers

- Define algorithm; state need for analysis (multiple solutions → pick efficient one)
- Define **time complexity** and **space complexity**
- Explain **best / worst / average** cases with one search or sort example
- Explain **asymptotic notations** O , Ω , Θ with definitions
- Show how to drop constants/lower terms ($3n+2 \rightarrow O(n)$)
- Mention structure rules: sequence sum, loop $\times n$, nested n^2 , if $\rightarrow \max$
- Then analyse **binary search** cases (see Unit 2 §4.2.3.3 worked content)

For Exam (Q39 lead-in)

Algorithm complexity is the study of an algorithm's time and space requirements as functions of input size. Time complexity measures how $T(n)$ grows; space complexity measures memory growth. Using asymptotic notations — Big- O (upper/worst), Big- Ω (lower/best) and Big- Θ (tight/average) — we compare algorithms for large n , ignoring machine-dependent constants. Analysis always considers best-case, worst-case and average-case inputs because the same algorithm may behave differently on different data.

Previously Asked Questions

- **Q39** (15 marks, Apr 2025) — Write a detailed note on algorithm complexity and analyse the best, worst, and average case complexities of binary search algorithm. (*Binary search case analysis is under 4.2.3.3.*)
 - *Answer:* Write in two parts:

Full paragraphs and table are under §4.2.3.3 below — use that for the complete 15-mark write-up.

1. **Algorithm complexity:** Time complexity = time as a function of input size n ; space complexity = memory as a function of input. Analyse best, worst and average cases. Use asymptotic notations Big- O (upper), Ω (lower), Θ (tight).
2. **Binary search cases** (sorted array only): Best **$O(1)$** if middle element is the key on first try; Worst **$O(\log_2 n)$** if key is absent or found after repeated halving; Average also **$O(\log_2 n)$** because each step discards half the list.

Unit 2: Searching and Sorting

4.2.1 Searching

Theory

Searching means finding a specific item in a collection of elements. The search is **successful** if the item is found; otherwise it is **unsuccessful**. Everyday analogies include finding a book on a shelf or a word in a dictionary. In data structures, the two techniques emphasised in this unit are **linear (sequential) search** and **binary search**.

Important Points

- Searching = locate a given item in a list/array

- Successful vs unsuccessful search
- Common techniques: **Linear search**, **Binary search**
- Efficiency matters: choose method that saves comparisons/time
- Array = contiguous collection of same-type elements, indices $0 \dots n-1$

Difference: Searching vs Sorting (PYQ Q23)

Searching	Sorting
Finds whether / where a key exists	Arranges elements in a definite order
Output: index / found-or-not	Output: ordered list
Does not (usually) rearrange data	Rearranges data by a criterion (asc/desc, key fields)
Examples: linear, binary search	Examples: selection, insertion, quick sort

For Exam

Searching is the process of finding a specific element in a set of elements; it succeeds if the element is found. Sorting is the process of arranging elements systematically according to a criterion (such as ascending order). Searching locates data; sorting organises data. Common searches are linear and binary search; common sorts in this unit are selection sort and insertion sort.

Previously Asked Questions

- **Q23** (2 marks, Apr 2025) — What is the difference between searching and sorting?
 - *Answer:* Searching finds whether / where a given key exists in a set of elements (e.g. linear or binary search). Sorting rearranges elements into a definite order according to a criterion such as ascending order (e.g. selection or insertion sort). Searching locates data; sorting organises data.

4.2.2 Linear Search (Sequential Search)

Theory

Linear (sequential) search checks each element of a list one by one from the start until the target is found or the list ends — like looking through an unsorted stack of books from top to bottom. It works on **sorted or unsorted** lists and is simple but slow for large n .

Sample in C

```
int linearSearch(int a[], int n, int item) {
    int i;
    for (i = 0; i < n; i++) {
        if (a[i] == item)
            return i;    /* found at index i */
    }
    return -1;           /* not found */
}
```

Important Points

- Checks elements **sequentially** from index 0
- Works on unsorted lists
- Returns index if found, else -1 / unsuccessful
- Disadvantage: high cost in worst case for large lists

Time complexity of linear / sequential search

Case	Situation	Complexity
Best	Key is first element (1 comparison)	O(1)
Worst	Key last or absent (n comparisons)	O(n)
Average	About $(n+1)/2$ comparisons	O((n+1)/2) $\approx$ O(n)

- **PYQ Q12 answer:** Best case of sequential search = **O(1)**

Example (SLM idea)

Search 25 in [92, 85, 53, 40, 25, ...] : compare 25 with 92, 85, 53, 40, then match at index 4.

For Exam

Linear (sequential) search examines each element of a list one by one until the target is found or the list ends. Best-case time complexity is $O(1)$ when the key is the first element; worst case is $O(n)$ when the key is last or absent; average case is about $O(n)$.

Previously Asked Questions

- **Q12** (1 mark, Apr 2025) — What is the best case time complexity of a sequential search?
 - *Answer:* $O(1)$ — when the key is the first element.

4.2.3 Binary Search

Theory

Binary search finds an item in a **sorted** list by repeatedly halving the search interval — like looking up a word in a dictionary by opening the middle, then choosing left or right half. Compare the key with the middle element: if equal, done; if key is smaller, search the left half; if larger, search the right half. Repeat until found or the interval is empty.

Prerequisite: the array **must be sorted**.

Sample in C

```
int binarySearch(int a[], int n, int target) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == target)
            return mid;          /* found */
        else if (a[mid] < target)
```

```

        low = mid + 1;      /* search right half */
    else
        high = mid - 1;     /* search left half */
    }
    return -1;              /* not found */
}

```

Important Points

- Works only on **sorted** lists
- Divide-and-conquer / half-interval / logarithmic search
- Each step discards half the remaining elements
- Much faster than linear search for large n

Worked example (Unit 2 style)

Sorted array of 7 elements; search $x = 4$.

1. Set `low` at first index, `high` at last; compute `mid`
2. If `x == arr[mid]`, return `mid`
3. If `x > arr[mid]`, set `low = mid + 1` (search right)
4. If `x < arr[mid]`, set `high = mid - 1` (search left)
5. Repeat until found or `low > high`

Another worked example (Unit 3 style)

List: `10, 12, 20, 32, 50, 55, 65, 80, 99`, search **12**:

1. Mid $\approx 50 \rightarrow 12 < 50 \rightarrow$ take left: `10, 12, 20, 32`
2. Mid = 12 $\rightarrow$ match $\rightarrow$ found at index 1

Search **80**:

1. Mid = 50 $\rightarrow 80 > 50 \rightarrow$ right: `55, 65, 80, 99`
2. Mid = 65 $\rightarrow 80 > 65 \rightarrow$ right: `80, 99`
3. Mid = 80 $\rightarrow$ found at index 7

For Exam

Binary search is an efficient searching technique for sorted arrays. It compares the key with the middle element and then searches only the left or right half, repeating until the key is found or the search space is empty. It follows the divide-and-conquer approach and requires a sorted list.

4.2.3.3 Time Complexity of Binary Search (worked — for Q39)

Theory

Because each unsuccessful comparison halves the problem size, the number of steps grows like $\log_2 n$ in typical and worst cases. Only when the middle element is the target on the first try do we get constant time.

Important Points — Best / Worst / Average (exam-ready)

Case	When it occurs	Comparisons (idea)	Complexity
Best	Target is exactly the first middle element	1 comparison	$O(1)$
Worst	Target absent, or found only after search space shrinks to one element	About $\log_2 n$ halvings	$O(\log_2 n)$
Average	Target found after several halvings (typical)	Also logarithmic	$O(\log_2 n)$

Why $O(\log_2 n)$?

Start with n elements. After 1 miss $\rightarrow \leq n/2$; after 2 $\rightarrow \leq n/4$; ... after k steps $\rightarrow \leq n/2^k$. Stop when size $\approx 1 \rightarrow n/2^k \approx 1 \rightarrow k \approx \log_2 n$.

Q39 combined answer outline (15 marks)

1. **Algorithm complexity:** define time/space; best/worst/average; asymptotic $O/\Omega/\Theta$; why analyse.
2. **Binary search:** definition + need for sorted array + brief algorithm.
3. **Cases:** best $O(1)$; worst $O(\log_2 n)$; average $O(\log_2 n)$ with short justification (halving).
4. Optional: one small numerical walk-through.

For Exam

Best-case time complexity of binary search is $O(1)$ when the target is the middle element on the first comparison. Worst-case complexity is $O(\log_2 n)$ when the element is missing or found only after the search interval is repeatedly halved down to one position. Average-case complexity is also $O(\log_2 n)$ because the search space shrinks logarithmically on typical inputs.

Previously Asked Questions

- **Q39** (15 marks, Apr 2025) — Write a detailed note on algorithm complexity and analyse the best, worst, and average case complexities of binary search algorithm.
 - *Answer:*
(Use the cases table and worked examples above for a complete answer.)
 - **Algorithm complexity:** It studies how much **time** and **space** an algorithm needs as the input size n grows. Time complexity is $T(n)$ (number of basic operations). Space complexity is the memory used. We study **best**, **worst** and **average** cases, and express growth using **Big-O**, Ω and Θ .
 - **Binary search:** Works only on a **sorted** array. Compare key with middle element; then search only the left or right half, repeating until found or search space is empty.
 - **Best case:** Key is the first middle element $\rightarrow O(1)$

- **Worst case:** Key missing, or found only after the interval shrinks to one element → about $\log_2 n$ comparisons → **$O(\log_2 n)$**
 - **Average case:** Also **$O(\log_2 n)$** , because each unsuccessful comparison halves the remaining elements.
-

4.2.4 Sorting

Theory

Sorting means arranging a group of items systematically according to a criterion (author, subject, age, ascending numeric order, etc.). In computing, sorting algorithms rearrange digital data — for example, student records by age. This unit details **selection sort** and **insertion sort**; Unit 3 adds **quick sort** as a divide-and-conquer sort (merge sort is named as another D&C example but not developed in detail in the SLM).

Important Points

- Sorting = arrange elements by a defined order/criteria
- Unit 2 focus: **Selection sort, Insertion sort**
- Unit 3: **Quick sort** (partition-exchange); merge sort mentioned as D&C example
- Goal: produce ordered data for easier search/processing

For Exam

Sorting is the process of arranging data systematically according to a given criterion. Important sorting methods in this block are selection sort, insertion sort and quick sort.

4.2.5 Selection Sort

Theory

Selection sort repeatedly finds the **smallest** remaining element and swaps it into the next position of the sorted prefix — like seating the shortest child first, then the next shortest, and so on. It is an **in-place** sort (little extra memory). Logic:

1. Find smallest in unsorted part
2. Swap with first unsorted position
3. Repeat until the array is sorted

Sample in C

```
void selectionSort(int A[], int n) {
    int i, j, index, temp;
    for (i = 0; i < n - 1; i++) {
        index = i;
        for (j = i + 1; j < n; j++) {
            if (A[j] < A[index])
                index = j;
        }
        temp = A[i];
        A[i] = A[index];
        A[index] = temp;
    }
}
```

```

    }
    /* swap A[i] and A[index] */
    temp = A[i];
    A[i] = A[index];
    A[index] = temp;
}
}

```

Important Points

- In-place; easy to understand
- Two nested loops → $O(n^2)$ for **best, average and worst** cases
- Uses few swaps: about $O(n)$ swaps (minimum among many sorts)
- Not efficient for large data sets
- Space complexity: $O(1)$

Example idea (SLM)

Unsorted integers → each pass selects minimum of remaining subarray and places it at position i ; after $n-1$ passes the array is sorted.

For Exam

Selection sort finds the minimum element in the unsorted portion and swaps it with the element at the current position, repeating until the list is sorted. Due to two nested loops its time complexity is $O(n^2)$ in best, average and worst cases, while extra space is $O(1)$.

4.2.6 Insertion Sort

Theory

Insertion sort works like sorting playing cards in hand. The array is virtually split into a **sorted** left part and an **unsorted** right part. Take the next key from the unsorted part and insert it into the correct position in the sorted part by shifting larger elements one place to the right.

Steps

1. Iterate i from 1 to $n-1$
2. `key = A[i]`
3. Shift predecessors greater than key to the right
4. Place key in the vacated position

Sample in C

```

void insertionSort(int A[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = A[i];
        j = i - 1;

```

```

        while (j >= 0 && A[j] > key) {
            A[j + 1] = A[j];
            j = j - 1;
        }
        A[j + 1] = key;
    }
}

```

Important Points

- Virtual split: sorted | unsorted
- Best case (already sorted): each key compared once → **$O(n)$**
→ **PYQ Q15 answer: $O(n)$**
- Worst case (reverse sorted): **$O(n^2)$**
- Average case: **$O(n^2)$**
- Space complexity: **$O(1)$**

Example (SLM): sort 7, 3, 11, 8, 5

1. Key=3 → shift 7 → 3, 7, 11, 8, 5
2. Key=11 → already in place → 3, 7, 11, 8, 5
3. Key=8 → shift 11 → 3, 7, 8, 11, 5
4. Key=5 → shift 11,8,7 → 3, 5, 7, 8, 11

For Exam

Insertion sort inserts each element into its correct position in the already-sorted left portion by shifting larger elements right. Best-case time complexity is $O(n)$ when the array is already sorted; worst-case and average-case complexities are $O(n^2)$. Space complexity is $O(1)$.

Previously Asked Questions

- **Q15** (1 mark, Apr 2025) — What is the best case time complexity of insertion sort?
 - *Answer:* $O(n)$ — when the array is already sorted (each key needs only one comparison).

Summary Table — Searching & Sorting Complexities

Algorithm	Best	Average	Worst	Extra space	Notes
Linear search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Unsorted OK
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Needs sorted array
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Few swaps
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Fast on nearly sorted

Algorithm	Best	Average	Worst	Extra space	Notes
Quick sort	$O(n \log n)^*$	$O(n \log n)^*$	$O(n^2)^*$	$O(\log n)$ stack*	*standard analysis; SLM focuses on method

Unit 3: Divide and Conquer Algorithms & Backtracking Algorithms

4.3.1 Divide and Conquer Algorithm

Theory

Divide and conquer is an algorithmic design pattern that solves a large problem by breaking it into smaller instances of the **same** problem, solving those recursively, and combining their solutions. Recursion stops at a **base case** small enough to solve directly. Two supporting ideas are the **relational formula** (recurrence describing cost) and the **stopping condition**.

Many famous algorithms use this pattern: binary search, merge sort, quick sort, maximum/minimum problems, Tower of Hanoi.

Three parts

1. **Divide** — break into smaller sub-problems
2. **Conquer** — solve sub-problems recursively (or directly if trivial)
3. **Combine** — merge sub-solutions into the full solution

Diagram (refer SLM): Fig. 4.3.1 — tree of sub-problems merging upward.

Important Points

- Pattern: Divide → Conquer → Combine
- Recursive case (large) vs base case (small/direct)
- Examples: binary search, quick sort, merge sort, Tower of Hanoi
- **Advantages:** simplifies hard problems; often faster; cache-friendly on small sub-problems; central to quick/merge sort
- **Disadvantages:** recursion overhead; may be harder than iteration; repeated overlapping sub-problems (needs memoisation); extra stack memory

For Exam

Divide and conquer divides a problem into smaller similar sub-problems, solves them recursively, and combines the results. The three phases are divide, conquer and combine. Recursion stops at a base case. Binary search, quick sort and merge sort are classic examples.

Divide and Conquer vs Backtracking (PYQ Q32)

Theory

Although both may use recursion and explore problem spaces, their goals differ. Divide and conquer **partitions** a problem into independent (or nearly independent) sub-problems whose

solutions are merged. Backtracking **builds a candidate solution step by step** and **abandons (backtracks from)** any partial path that cannot lead to a valid complete solution, then tries another choice.

Important Points — Differentiation table

Aspect	Divide and Conquer	Backtracking
Core idea	Split → solve parts → combine	Try choices; undo when stuck
Sub-problems	Smaller instances of same problem	Paths in a state/search tree
Combination	Explicit combine/merge step	No merge of independent answers; explores alternatives
Typical use	Sorting, searching (quick, merge, binary)	Puzzles, mazes, constraint satisfaction, games
Example in SLM	Binary search, Quick sort	Mini-max game tree
Failure handling	Base case solves directly	Invalid path → backtrack to last valid point
Completeness of search	Solves structured sub-instances	Systematically considers potential solutions

For Exam

Divide-and-conquer algorithms break a problem into smaller similar sub-problems, solve them recursively, and combine the solutions (e.g., quick sort, binary search). Backtracking algorithms build a solution incrementally and abandon any path that fails the constraints, returning to the previous choice point to try another option (e.g., maze solving, mini-max). Divide and conquer emphasises partition-and-merge; backtracking emphasises trial-and-error with undo.

Previously Asked Questions

- **Q32** (4 marks, Apr 2025) — Differentiate between divide and conquer algorithms and backtracking algorithms.
 - *Answer:*

Divide and Conquer	Backtracking
Breaks a problem into smaller similar sub-problems	Builds a solution step by step (incrementally)
Solves sub-problems (often recursively) and combines results	If a choice fails constraints, undo it and try another path
Emphasises partition and merge	Emphasises trial-and-error with undo
Examples: Binary search, Quick sort, Merge sort	Examples: Maze solving, Mini-max game tree, N-queens style puzzles

In short: divide and conquer = **split-solve-merge**; backtracking = **try-check-undo**.

4.3.2.1 Binary Search (as Divide and Conquer)

Theory

Binary search is a divide-and-conquer search on a sorted list: compare with middle, then recursively/iteratively search only one half. Implementation steps (SLM): read key → find mid → compare → if unequal, repeat on left or right sub-list → continue until found or one element left unmatched.

Important Points

- D&C view: divide list at mid; conquer one half; no heavy combine (answer is an index)
- Limitation: **unsorted lists cannot use binary search**
- Also called half-interval / logarithmic search

For Exam

Binary search applies divide and conquer on sorted data by comparing the key with the middle element and continuing the search in only one half until the element is found or the list is exhausted.

4.3.2.2 Quick Sort Algorithm

Theory

Quick sort (Tony Hoare, 1959), also called **partition-exchange sort**, is a fast divide-and-conquer sorting method. It selects a **pivot**, **partitions** so that elements less than the pivot are on the left and greater on the right (pivot then in final position), and **recursively** sorts the two sides.

Three steps

1. **Pivot selection** — often leftmost or rightmost element of the current portion
2. **Partitioning** — reorder around pivot
3. **Recur** — quick-sort left and right sub-arrays

Partition idea (SLM, pivot = last element)

Array `{10, 80, 30, 90, 40}`, pivot **40**:

- Elements ≤ 40 move left; greater stay right
- After first partition: left of 40 are smaller, right are greater
- Recurse on both sides until fully sorted

Sample in C (simple outline)

```
/* Place pivot, then sort left and right parts */
void quickSort(int A[], int low, int high) {
    int p;
    if (low < high) {
        p = partition(A, low, high); /* pivot final position */
        quickSort(A, low, p - 1);
        quickSort(A, p + 1, high);
    }
}
```

```
}  
}
```

(Exam tip: write the three steps — choose pivot, partition, recurse — with a small numeric example; full partition code is optional.)

Important Points

- Divide-and-conquer sorting via partitioning
- Heart of algorithm = **partition**
- After partition, pivot is in **final sorted position**
- Often much faster in practice than simple $O(n^2)$ sorts on average
- Named “quick” because it is typically 2–3× faster than many standard sorts (SLM)

For Exam

Quick sort is a divide-and-conquer algorithm that selects a pivot, partitions the array so smaller elements lie left and larger elements lie right of the pivot, then recursively sorts the two partitions. It is also called partition-exchange sort.

4.3.3 Backtracking

Theory

Backtracking solves problems by exploring candidate solutions **incrementally**. At each step a choice extends the current path. If the path violates constraints or cannot succeed, the algorithm **backtracks** to the last valid decision point and tries a different choice. It is powerful for combinatorial problems: puzzles, mazes, and constraint satisfaction.

Important Points

- Incremental construction of solutions
- Abandon invalid partial solutions (prune)
- Systematic exploration of the search space
- Useful when many potential combinations exist

For Exam

Backtracking explores possible solutions step by step and abandons any path that fails to satisfy the problem constraints, returning to the previous choice to try another alternative until a valid solution is found or all options are exhausted.

4.3.3.1–4.3.3.2 Mini-Max Algorithm

Theory

Mini-max is a recursive **backtracking / decision-making** algorithm for turn-based two-player games. One player is the **maximiser** (wants highest score); the other is the **minimiser** (wants

lowest score). Assuming both play optimally, mini-max chooses the best move for the current player.

Definition (SLM): recursive/backtracking algorithm used in decision-making and game theory that provides an optimal move assuming the opponent also plays optimally.

Workflow (game tree)

1. Generate game tree; assign utility values to terminal nodes. Maximiser initial value $-\infty$; minimiser $+\infty$.
2. At maximiser nodes, take **max** of children.
3. At minimiser nodes, take **min** of children.
4. Back up values to the root; root's value and chosen child give the optimal first move.

SLM numeric sketch

- Maximiser layer: $D=4$, $E=6$, $F=-3$, $G=7$
- Minimiser: $B=\min(4,6)=4$; $C=\min(-3,7)=-3$
- Root maximiser: $A=\max(4,-3)=4$

Important Points

- Maximiser $\leftrightarrow$ maximum benefit; Minimiser $\leftrightarrow$ minimum for opponent
- Positive board score $\rightarrow$ maximiser advantage; negative $\rightarrow$ minimiser
- Performs depth-first exploration of the game tree
- Drawback: becomes slow for complex games (chess, Go)

Diagram (refer SLM): Figs. 4.3.18–4.3.21 — stages of mini-max backup.

For Exam

The mini-max algorithm is a recursive backtracking method for two-player games. The maximiser chooses moves that maximise the score and the minimiser chooses moves that minimise it. Utilities at leaf nodes are backed up alternately by max and min operations to select an optimal move at the root, assuming optimal play by both sides.

Unit 4: Minimum Cost Spanning Trees

4.4.1 Concepts of Spanning Trees

Theory

Graphs model pairwise relationships (cities and roads, houses and cable lines). A **spanning tree** of a connected graph G is a subgraph that includes **all vertices** of G and enough edges to connect them **without cycles**. If G has n vertices, a spanning tree has exactly **$n-1$** edges. The cable-TV analogy: connect all houses with routes but avoid loops that waste money.

BFS/DFS traversal trees are spanning trees of the explored connected graph. A graph may have **many** spanning trees.

Important Points

- Spanning tree = connected acyclic subgraph containing **all** vertices
- Edges in spanning tree = $n - 1$ for n vertices
- No cycles; removing any edge disconnects it; adding any edge creates a cycle
- Every connected undirected graph has ≥ 1 spanning tree
- Disconnected graph $\rightarrow$ **no** spanning tree
- Complete undirected graph: up to n^{n-2} spanning trees (Cayley's formula; SLM writes n^{n-2})
- Applications: network planning (water, phone, electric), clustering, routing

Diagram (refer SLM): Fig. 4.4.2 — one graph and several spanning trees.

For Exam

A spanning tree of a connected graph is a tree-shaped subgraph that contains all vertices and $n-1$ edges with no cycles. It connects every node with a minimal set of links. Connected undirected graphs have at least one spanning tree; disconnected graphs have none.

4.4.2 Minimum Spanning Tree (MST)

Theory

When edges have **weights** (costs), different spanning trees have different total costs. A **Minimum Spanning Tree (MST)** is a spanning tree whose total edge weight is **as small as possible**. Cable example: connect all houses so the sum of connection costs is minimal (SLM example sum 103). Another SLM example: spanning trees of costs 15, 13 and 11 $\rightarrow$ MST cost **11**.

Applications: telecom networks between cities, water/electrical grids, map path design.

Important Points

- MST = spanning tree with **minimum total edge weight**
- Cost of a tree = sum of its edge weights
- Graph must be connected, undirected, weighted
- A graph can have multiple MSTs if ties exist, but same minimum cost
- Used to design cheapest connecting networks

Diagram (refer SLM): Fig. 4.4.3–4.4.4 — weighted graph and MST examples.

For Exam

A minimum spanning tree of a connected weighted graph is a spanning tree for which the sum of edge weights is minimum. Its cost is that total weight. MSTs are used to design low-cost networks that still connect all nodes.

4.4.3 Prim's Algorithm

Theory

Prim's algorithm builds an MST by growing a single tree from an arbitrary start vertex. At each step it adds the **least-weight edge** that connects a vertex **already in the tree** to a vertex **outside** the tree (no cycle). It stops when all vertices are included.

Algorithm (SLM)

1. Choose any start node; put it in the spanning tree
2. Collect incident edges from tree vertices to new vertices
3. Pick the least-weight edge in that set
4. If it forms no cycle, add it; else discard and pick next least
5. Repeat until all nodes are in the tree

Important Points

- Requires weighted, connected, **undirected** graph
- Grows **one tree** from a root (vs Kruskal's forest)
- Always add cheapest safe edge out of the current tree
- Never introduce cycles

Worked outline (SLM Prim example — 7 nodes)

Start at **A**:

1. Add **(A,B)** (least from A)
2. Add **(B,C)**
3. Add **(B,E)**
4. Add **(E,D)**
5. Add **(D,G)**
6. Add **(G,F)**

MST edges: (A,B), (B,C), (B,E), (E,D), (D,G), (G,F)

Minimum cost = 17 (sum of six edge weights)

Diagram (refer SLM): Figs. 4.4.5–4.4.12 — step-by-step Prim growth.

For Exam

Prim's algorithm finds an MST by starting at any vertex and repeatedly adding the smallest-weight edge that joins a new vertex to the tree without forming a cycle, until all vertices are included. The graph must be connected, undirected and weighted.

4.4.4 Kruskal's Algorithm

Theory

Kruskal's algorithm builds an MST by considering edges in **increasing order of weight**. It begins with a **forest** of n single-node trees. An edge is added only if its endpoints lie in

different trees (components); if both ends are already in the same tree, the edge would form a cycle and is rejected. Sorted edges are commonly kept in a **priority queue**.

Algorithm (SLM)

1. Make each vertex its own tree (forest of n trees)
2. Sort all edges by ascending weight
3. Store them in a priority queue
4. For each edge in order:
 - If it creates a cycle → reject
 - Else add it (merge two trees)
5. Stop when the forest becomes one tree with $n-1$ edges (or edges exhausted)

Important Points

- Starts as **forest of trees**, not one growing tree
- Process edges **lightest first**
- Add edge iff it joins **different** components
- Reject edge if it creates a **cycle**
- Final MST cost = sum of accepted edge weights
- **PYQ Q34**: describe with example (full worked content below)

Prim vs Kruskal (quick compare)

Prim	Kruskal
Grows one tree from a start node	Grows a forest; merges components
Pick cheapest edge leaving the tree	Pick cheapest edge overall that is safe
Needs connected graph from the start	Naturally handles edge list / components

For Exam

Kruskal's algorithm finds a minimum spanning tree by sorting edges in increasing order of weight and adding an edge only when it does not form a cycle, i.e., when its endpoints belong to different trees in the forest. It starts with n single-node trees and merges them until one MST remains.

Previously Asked Questions

- **Q34** (4 marks, Apr 2025) — Describe Kruskal's algorithm with an example.
 - *Answer*: Kruskal's algorithm finds a **Minimum Spanning Tree (MST)** of a connected weighted undirected graph.

Steps:

1. Sort all edges in increasing order of weight.
2. Start with each vertex as a separate tree (a forest).

3. Take the next cheapest edge. If its two ends belong to different trees (no cycle), **add** it; otherwise **reject** it.
4. Repeat until the tree has $(n - 1)$ edges (all vertices connected).

Example (from notes): Accepted edges (E,G), (D,E), (A,B), (A,D), (E,F), (B,C) give MST cost $1+2+3+5+7+9 = 27$. Full accept/reject table is given below.

Kruskal's Algorithm — Worked Example (SLM-style, for Q34)

Graph: 7 nodes (A–G), 12 weighted edges (same family of example as SLM Figs. 4.4.13–4.4.27).

Step 0 — Initialise

- Forest: {A} {B} {C} {D} {E} {F} {G}
- Sort edges ascending into a priority queue (lightest first).
Illustrative order used in SLM walk-through:
(E,G)=1, (D,E)=2, (A,B)=3, (D,G)=4, (A,D)=5, (B,E)=6, (E,F)=7, (F,G)=8, (B,C)=9, (C,F)=11, (B,D)=13, (C,E)=15

Step-by-step decisions

Step	Edge	Weight	Cycle?	Action
1	(E,G)	1	No	Add — merge E,G
2	(D,E)	2	No	Add — merge D with E-G
3	(A,B)	3	No	Add — merge A,B
4	(D,G)	4	Yes (D-E-G already linked)	Reject
5	(A,D)	5	No	Add — connect A-B side to D-E-G
6	(B,E)	6	Yes	Reject
7	(E,F)	7	No	Add — bring F in
8	(F,G)	8	Yes	Reject
9	(B,C)	9	No	Add — bring C in
10	(C,F)	11	Yes	Reject
11	(B,D)	13	Yes	Reject
12	(C,E)	15	Yes	Reject

Accepted MST edges: (E,G), (D,E), (A,B), (A,D), (E,F), (B,C)

Minimum cost = $1+2+3+5+7+9 = 27$

Diagram (refer SLM): Figs. 4.4.14–4.4.27 — forest/priority-queue snapshots; Fig. 4.4.27 final MST.

Exam write-up tip (4 marks)

1. State steps (sort edges; add if no cycle).
2. Show 4–6 lines of edge accept/reject.

3. List final edges and total cost.

Quick Revision Sheets

Block 1 Quick Revision Sheet

Topic	One-line cue
Data structure	Organisation of data for efficient use
Linear vs non-linear	Sequence vs hierarchy/network
Contiguous example	Array
Static vs dynamic memory	Compile-time fixed vs run-time heap (<code>malloc</code> / <code>free</code>)
Stack	LIFO; push/pop at top
Queue	FIFO; enqueue rear, dequeue front
Circular queue	Ring; reuse space
Deque	Insert/delete at both ends
Priority queue	Serve by priority
Infix → Prefix	Reverse → convert → reverse
Postfix evaluation	Push operands; pop-operate-push
<code>free()</code>	Release dynamic memory
Recursion	Function calls itself with base case
Tower of Hanoi	Move $n-1$ aside → move largest → move $n-1$; moves = $2^n - 1$

Block 2 Quick Revision Sheet

Operation	Key idea
Node	Data + link (self-referential struct)
Traverse (singly)	Temp = Start; while Temp ≠ NULL; Temp = Temp → link
Insert begin	new → link = Start; Start = new
Insert end	Walk to last; last → link = new; new → link = NULL
Insert at POS	prenode → link = new; new → link = postnode
Delete begin	Start = Start → link; free old
Delete end	second-last → link = NULL; free last
Delete at POS	prenode → link = postnode; free POS
Search	Linear scan from Head until match or NULL
Circular LL	Last → link = Start; no NULL end
Doubly LL	Prev + Next; traverse both ways
Stack (LL)	Push/pop at front (Top)
Queue (LL)	Enqueue at rear; dequeue at front

Operation	Key idea
LL vs Array	Dynamic / non-contiguous vs fixed / contiguous

Block 3 Quick Revision Sheet

Topic	Must remember
Degree of node (tree)	Number of children
Height / Depth	Longest root-leaf / path length from root (root = 0)
Binary tree types	Complete, Full, Perfect, Balanced, 2-tree
Traversals	Inorder LVR, Preorder VLR, Postorder LRV
BST	Left < node < Right; inorder sorted; insert/search/delete
AVL	Self-balancing BST; $BF = H_L - H_R \in \{-1, 0, 1\}$; LL / RR / RL / LR
B-tree	Multi-key, multi-child, disk-oriented; all leaves same level
Graph	$G = (V, E)$
Multi-graph	Self-loop and/or parallel edges
BFS / DFS	Queue / Stack
Representations	Adjacency matrix & adjacency list
Graph applications	TSP, maps, social networks, recommendations

Block 4 Quick Revision Sheet

Topic	Must-remember
Algorithm properties	Input, Output, Definiteness, Finiteness, Effectiveness
Time complexity	Time as function of input size n
Space complexity	Memory as function of input
Cases	Best / Worst / Average
Big-O / Ω / Θ	Upper / Lower / Tight
Q35 answers	$O(1)$, $O(n)$, $O(n^5)$, $O(n^2)$
Sequential search best	$O(1)$
Insertion sort best	$O(n)$
Binary search	Best $O(1)$; Avg/Worst $O(\log_2 n)$
Selection sort	Always $O(n^2)$; space $O(1)$
D&C vs Backtracking	Split-merge vs try-and-undo
Quick sort	Pivot + partition + recurse
MST	Spanning tree of minimum total weight
Prim	Grow one tree from a start vertex
Kruskal	Sort edges; add if no cycle; forest $\rightarrow$ tree